A Guide to the PC-LISP Interpreter

Peter Ashwood-Smith

University of Toronto

Table of Contents

1. Introduction

PC-LISP is a small implementation of LISP for just about any machine with a good C compiler. This manual is biased towards the UNIX and MS-DOS versions.

While small, it is capable of running a pretty good subset of Franz LISP. The functions are supposed to perform in the same way as Franz with a few exceptions made for effeciencies sake. Version 2.16 has the following features.

- Types fixnum,flonum,list,port,symbol,string, hunk, array. Forms lambda, nlambda, macro and lexpr.

- Read Macros including splicing read macros.

- Full garbage collection of ALL types.

- Compacting relocating heap management.

- Access to some MSDOS BIOS graphics routines.

- Over 160 built in functions, sufficient to allow you to implement many other Franz functions in PC-LISP.

- Stack overflow detection & full error checking on all calls, tracing of user defined functions, and dumping of stack via (showstack).

- One level of break from which bindings at point of error can be seen.

- Reasonable size, requires minumum of 300K (machine RAM required may differ depending on OS size).

- Access to as much (non extended) memory as you've got and control over how this memory is spread among the various data types.

This program is Shareware. This means that it you are free to distribute it or post it to any BBS that you want. The more the better. The idea is that if you feel you like the program and are pleased with it then send us $15 to help cover development costs. Source code for this program is available upon request. You must however send me 3 blank diskettes and about $1.50 to cover first class postage. The program can be compiled with any good C compiler that has a pretty complete libc. In particular the program will compile with almost no changes on most UNIX systems. A source code guide will probably be included with the source if it is finished at the time I receive your source request. If you send diskettes, SEND NEW, GOOD QUALITY DISKS as I have had problems writing IBM-PC readable data to old or poor quality diskettes with my Tandy 2000's 720K disk drives.

2. A Warning

PC-LISP is distributed as ShareWare. The executable and source code may be freely distributed. It is contrary to the purpose of ShareWare to charge more than media and or mailing costs for this program in any form source,disk,tape etc. If you use PC-LISP you do so at your own risk. I will not be held responsible for loss or dammage of any kind as a result of the correct or incorrect use of this program. If you modify the source and redistribute this source or its resulting executable I ask that you add a "modified by x" or a "ported to z by y" line to the initial banner and comment the code accordingly. Please do not remove my name from the banner.

3. A Note

The rest of this manual assumes some knowledge of LISP, MSDOS/UNIX and a little programming experience. If you are new to LISP or programming in general you should work your way through a book on LISP such as LISPcraft by Robert Wilensky. You can use the interpreter to run almost all of the examples in the earlier chapters. I obviously cannot attempt to teach you LISP here because it would require many hundreds of pages and there are much better books on the subject than I could write. Also, there are other good books on Franz LISP besides LISPcraft.

4. If You Want To Try Pc-Lisp Right Now

Make sure that PC-LISP.EXE and PC-LISP.L are in the same directory. Then type PC-LISP from the DOS prompt. Wait until you

```
get the "-->" prompt. Here is what you should see starting by
```

typing pc-lisp at the prompt:

PC-LISP V2.16 Copyright (C) 1986 by Peter Ashwood-Smith NNN cell bytes, NNN alpha bytes, NNN heap bytes --- [pc-lisp.l] loaded ---

-->

Be patient, it takes a few seconds to load the program especially off a floppy. When you see the first line with the version number it will take another second or two to produce the status line. (The N's depend on how much memory you have). At this point PC-LISP is up and running and is reading LISP from the file PC-LISP.L. Again this takes a second or two.

If your machine has some sort of graphics capability you can try the graphics demo as follows. Type "(load 'turtle)" without

the "'s. Wait until you see the "t" and the prompt "-->" again,

then type "(GraphicsDemo)". You should see some Logo like squirals etc. If you do not have any graphics capability try "(load 'queens)" or "(load 'hanoi)" and then (queens 5) or (hanoi 5) respectively. For a more extensive example turn to the last couple of chapters in LISPcraft and look at the deductive data base retriever. Type (load 'match) and look at the match.l documentation.

5. Example Load Files And The Pc-Lisp.L File

Included with PC-LISP (V2.16) are a number of .L files. These include: PC-LISP.L, MATCH.L, TURTLE.L, DRAGON.L, DIFF.L and perhaps a few others. These are as follows.

6. Pc-Lisp.L

A file of extra functions to help fill the gap between PC and Franz LISP. This file defines the pretty print function and a number of macros etc. It will be automatically loaded from the current directory or from the directory whose path is set in LISP_LIB when PC-LISP is executed. The functions in this file are NOT documented in this manual, look instead at a Franz manual.

7. Match.L

A small programming example taken from the last 2 chapters of LISPcraft. It is a deductive data base retriever. This is along the lines of PROLOG. Very few changes were necessary to get this to run under PC-LISP.

8. Turtle.L

Turtle Graphics primitives and a small demonstration program. To run the demo you call the function "GraphicsDemo" without any parameters. This should run albeit slowly on just about every MS-DOS machine. The graphics primitives look at the global variable !Mode to decide what resolution to use. If you have mode 8 (640X400) you should use it as the lines are much sharper. Turtle graphic modes can be set by typing (setq !Mode - number-). Have a look at TURTLE.L to see how they work.

9. Dragon.L

A very slow example of a dragon curve. This one was translated from a FORTH example in the April/86 BYTE. It takes a long time on my 8Mhz 80186 machine so it will probably run for a few hours on a PC or AT. I usually let it run for about 1/2 hour before getting tired of waiting. To run it you just type (load 'dragon) then type (DragonCurve 16). If you have a higher resolution machine like a Tandy 2000 then type (setq !Mode 8) before you run it and it will look sharper at this (640x400) resolution.

10. Diff.L

Is an example of symbolic computation. It takes a simple expression and computes it's first, second, third, fourth and fifth symbolic derivative. Again this is just a small example that should not be taken too seriously in itself.

11. Users Guide

The PC-LISP program is self contained. To run it just type the command PC-LISP or whatever you called it. When it starts it will start grabbing memory in chunks of 16K each. By default PC- LISP will grab 50 blocks but by setting the LISP_MEM environment variable this can be controlled. Note, there is a hard limit of 75 blocks. The LISP_MEM environment variable is set in MS-DOS or UNIX as follows:

set LISP_MEM=(28B,4A,4H)

Which means allocate up to 28 blocks total, of which 4 are for alpha objects and 4 are for heap objects. The remainder go for cons cell, file, array base, flonum and fixnum objects. By default PC-LISP will allocate up to 50 blocks. 1 of which is dedicated for alpha and 1 for heap. Note the environment variable MUST be formatted as above. No spaces are permitted, the brackets must be present as must the B,A and H (all capitals) after the block counts.

After allocating memory PC-LISP will then print the banner message followed by the actual amount of memory allocated for each of the three basic object types. Next, before processing the command line, PC-LISP will look for a file called "pc-lisp.l" first in the current directory, next in the library directories specified in the LISP_LIB environment variable as per the (load) function. If it finds pc-lisp.l it will read and evaluate commands from this file until the end of file is reached. Finally PC-LISP will read the parameters on the command line. The command line may contain any number of files eg:

PC-LISP file file .... file

The files on the command line are processed one by one. This consists of loading each file as per the (load) function. This means that PC-LISP will look in the current directory for 'file', then in 'file'.l, then in the directories given in the LISP_LIB environment variable, when found the file is read and every list is evaluated. The results are NOT echoed to the console. Finally when all the files have been processed you will find yourself

```
with  the PC-LISP top level prompt '-->'.  Typing  control-Z  and
```

ENTER (MS-DOS end of file) or CONTROL-D (UNIX end of file) when

```
you  see the '-->' prompt will cause PC-LISP to exit to  whatever
```

program called it. If an error occurs you will see the prompt

```
'er>'.  For more info see the 'TERMINATION OF EVALUATION' section
```

of this manual and the commands (showstack), (trace), and (untrace).

SYNTAX OR WHAT IS A LIST ANYWAY? ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

You will now be in the PC-LISP interpreter and can start to play with it. Basically it is expecting you to type an S- expression whose value it will evaluate and return. Formally an S-expression can be defined with a B.N.F Grammar where + means at least one occurence of and, * means any number of occurences of.

<S-expression> ::= <fixnum> | <flonum> | <string> | <symbol> | '(' <elements> ')'

+ <elements> ::= (<S-expression>) '.' <S-expression> * | (<S-expression>)

Where characters whose ascii values are in 0..31 are ignored and have no effect other than delimiting other input items. Also characters between ; and the end of a line are ignored in the same way as the white space characters just described, these are used to introduce comments into your LISP programs.

The the basic list elements <fixnum>, <flonum>, <string> and <symbol> are defined as follows.

A <fixnum> is a sign + , - or none followed by a sequence of digits 0..9. If the sequence of digits represents a fixnum larger than can be stored in a 32 bit integer it is taken to be the nearest <flonum>. A <fixnum> can always be spotted when it is printed by the lack of a radix point. Examples are: 2, +2, -2, and -333333 .

A <flonum> is a sign + , - or none followed by digits 0..9 which may be followed by a radix point and more digits 0..9 this may optionally be followed by an exponent specifier 'e' or 'E' which may optionally be followed by a sign + , - or none, optionally followed by the exponent digits 0..9. A <flonum> can always be spotted when it is printed by the presence of either a radix point, or the exponent specifier 'e'. Examples : 2.0, -2.0, +2.0, -2e10, -2e+20, -4.0E-13, 2E, -2E

A <string> is a " followed by up to 254 characters followed by a terminating " or |. If the character \ is present in the string and the following character is one of t,b,n,r or f the two characters are replaced by a tab, backspace, newline, carriage return or form feed respectively. If the \ is not followed by one of the previously mentioned special characters, the following character is used to subtitute the \ and itself in the string. The \ is called the escape character and allows you to put non printing formatting characters into a string. It also allows you to put a " or | into a string which you could not otherwise do. Examples: "abcd", "a\tb", "a\"b", "a\|b".

SYNTAX OR WHAT IS A LIST ANYWAY? CONT'D ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ A <symbol> is either a string delimited with |'s instead of the "'s, or a sequence of characters none of which are spaces or non printing characters with ascii values < 32 or > 126. A \ may be used to escape the following character just as in a string but is also legal without the delimiters. If not delimited the character after the escape is taken literally rather than translated to a newline etc. If delimited any character may be placed between the | delimiters with the exception of " or | which must be preceeded by the escape character if they are to be literally included in the symbol. If the symbol is not delimited by |'s and does not contain an escaped character then the characters must be in a sequence that follows the following rules. The characters ( ) [ ] " | and ; are reserved and will cause termination of the symbol. The set of characters that are skipped as white space (those with ascii values in the range 0..31) are termed white space characters. The set of characters that have been defined as read macros are termed macro trigger characters. Only the ' char is initially a read macro trigger character. The special characters are all of these above character classes. Using these definitions, a symbol can either start with a character in 0..9 or a character not in 0..9. If the character is not in 0..9 then the the following characters can be chosen from among all but the special characters. If the first character in the symbol is in 0..9 then the last character must be chosen from among the set of all characters that are neither special nor in 0..9. A symbol may be composed of up to 254 characters all of which are significant. Here are a few examples: \(a1 1a 1- 1234abc #hi# !hi% An_ATOM |ab\nc| junk.l ThisIsOneRatherLargeAtomThatDemonstratesLength \1 2e1\0

An atomic S-expression is just one of a fixnum, flonum, string and symbol. The only other type of S-expression that can be input is a list S-expression.

In order to describe what a list S-expression is you need to know some lisp terminology for the parts of a list. First a list consists of two parts, the first element of the list is called the car of the list and the rest of the elements in the list is called the cdr of the list. For example the list (a b c) has car a and cdr (b c). Now that we know the two parts of a list, we need to know how to build a list. A list is built with a cons or constructor cell. The constructor cell has two parts to it, the first is the car of the list and the second is the cdr of the list. Hence one cons cell describes one list. Its car part describes the first element in the list, and its cdr part describes the list of the rest of the elements in the list. For the example list (a b c), the internal structure may look

```
something  like this:  (where a [ | ] represents a cons cell *-->
```

is a pointer, / is a nil pointer)

```
[*|*] ---> [*|*] ---> [*|/]
```

| | |a b c

SYNTAX OR WHAT IS A LIST ANYWAY? CONT'D ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

Here is an example of a simple nested list which can be input as : (a (b c) nil d) and which results in a structure like this:

```
[*|*] ---> [*|*] ---> [/|*] ---> [*|/]
```

| | |v v v

```
  a        [*|*] ---> [*|/]       d
```

| |v v b c

The dot '.' can be used to separate the last element in a list from the others in the list. When this occurs the constructed list will have a slightly different last cons cell second field. Rather than pointing to another cons cell whose car points to the last element, this field will point directly to the last element. For example inputting (a . b) creates the following list structure, which will also print as (a . b).

[*|*] | | v v a b

However if the last element in the list is another list and we preceed it by a dot, the list is spliced into the upper list as if the last element were not really a list. For example if I were to input (a . (b . (c))) the following structure which is identical to that constructed by (a b c) would be built. It will also print as (a b c).

```
[*|*] ---> [*|*] ---> [*|/]
```

| | |v v v a b c

The dotted pair is not normally used except when you wish to save storage. An example might be when you create a list of symbols and their associated values. In this case making the symbol and its associated value a dotted pair will save 1 cons cell or about 10 bytes per symbol value pair.

Finally, I have shown these structures with symbol elements. You can have absolutly any type as an element of a list, including of course a list as shown in the second example above. This is a very quick look at list structure and you should look at LISPcraft for more details.

12. Meta Syntax

Following are some syntactic properties that are really above the level of the syntax of a simple S-expression. Thus they are called meta syntax conventions. I consider Meta syntax as anything that does not conform to the B.N.F grammar previously given. These extensions to the syntax of S-expressions consist of any extra syntax intdoduced by built in or user defined read macros and the replacement of multiple parenthesis which occurs when a single super parenthesis is used.

PC-LISP supplies one built in read macro called 'quote' and written using the little ' symbol. This read macro is just a short hand way of writing the list (quote S). Where S is the S- expression that follows the ' in the input stream. Here are some examples of the simple conversion that the read macro performs on your input.

```
'apples          -- goes to -->     (quote apples)
```

'|too late| (quote |too late|) '(1 2 3) (quote (1 2 3)) "a (quote (quote a)) '"hi" (quote "hi")

If you are new to LISP you will soon see just how useful this little read macro is when you start typing expressions. It reduces the amount of typing you must do, reduces the amount of list nesting you have to look at and draws attention to data in your expressions.

User defined read macros are also provided. See the (setsyntax) function in the next section of the manual. The backquote macro together with comma (,) and at (@) are implemented in the PC-LISP.L load file, but are not documented here. Again, see LISPcraft for a discussion of these read macros.

PC-LISP also provides the meta or super parenthesis [ ]. One of the problems with LISP is the often overwhelming number of parenthesis. It is very common to not supply enough closing)'s and therefore have syntactic/semantic errors in your program. The [and] characters when properly used allow you to force certain structures even if enough)'s have not been provided. They operate as follows. When the [is encountered in the input, it acts like a (except that a note is made of the number of unclosed ('s so far. Now when a] is encountered in the input, all lists up to and including the matching [are closed. If there is no matching [, ie none has been entered or all have been closed with a] then all open lists are closed. These parenthesis may be nested up to 16 levels deep. But, deep nesting reduces their usefullness. NOTE: If you open a list with a [you must close it with a]. If you close it with a) you will cause the next [ ] pair to function incorrectly. The super nesting information is reset whenever a new file is processed, or whenever the break level is entered. That is, meta parenthesis cannot be used accross a load or read of another file. Finally, here are a few example legal inputs which use the meta parenthesis and the list that results from their input.

```
(("hello world\n"] -- goes to --> (("hello world\n")))
```

(([(((8 9] 10] ((((((8 9)))) 10)) [[[[[a]]]]] (((((a)))))

I should just mention again the fact that meta parenthesis will not operate accross multiple reads. For example suppose you were using (read) to get sublists from lists in one file, and then switched to reading lists from another file, then returned to the original file. If the original input file made use of the super parenthesis and the particular sublist being read was between a pair of superparenthesis, this information would be lost when you

resume reading the file. Hence the next] you hit will terminate all open lists rather than those opened after the lost [. The moral of this example is not to use the super parenthesis in a data file whose reading may be interrupted by other I/O. This is not a particularly imposing limitation.

13. A Franz Difference

PC-LISP V2.16 is different from Franz in how the \ character is interpreted when followed by n,t,r etc. in a string or | delimited symbol. Franz does not convert them to newline, tab, carriage return etc. Instead, Franz simply takes the next character literally. You can override the 'smart-backslash' by using (sstatus) to set the option to nil. The smart backslash is much more convenient though because you can say (patom "stuff\n") instead of (patom "stuff") (terpri). It is however non portable so don't use the smart-backslash unless you are only writing for PC-LISP.

14. Syntax Errors

When you enter a list which is not correct syntactically the interpreter will return the wonderfully informative 'syntax error' message. This message may be followed by a message as to the cause such as 'atom too big' or it may be followed by a pretty print of an expressopm which was close to where the error was detected. You will have to figure out where it is in the input list. Note that if you do not finish entering a list, ie you put one too few closing)'s on the end, the interpreter will wait until you enter it before continuing. If you are not sure what has happened just type "]]" and all lists will be closed and the interpreter will try to do something with the list. If you are running input from a file the interpreter will detect the end of file and give you a 'syntax error' because the list was unclosed. Try also (showstack), it can help pinpoint the error in a large load file. V2.16's syntax error handling could be improved.

15. Evaluating S-Expressions

The interpreter expects an S-expression to be typed at the

```
prompt  '-->'.  The interpreter will evaluate the expression  and
```

print the resulting S-expression. If the expression is either a fixnum or a flonum, the interpreter just returns it because a number evaluates to itself. If the expression is a string, the interpreter also returns it because a string evaluates to itself. If however the expression is a symbol, the interpreter returns the binding of the symbol. It is an error to try to evaluate a symbol that has no binding. Certain predefined atoms are prebound, while all other symbols are unbound until bound by a function call or a set / setq. If the expression is a list, then the first element in the list is taken to be a function name or description, the rest of the elements are taken to be parameters to the function. The interpreter will normally evaluate each of the arguments and then pass them to the appropriate function whose result is returned. For example: The list S-expression with a '+' as the first element and fixnums as elements will evaluate as the sum of the fixnums. Eg.

```
-->(+ 2 4 6 8)
```

20

We can also compose these function calls by using list nesting. Sublists are evaluated prior to upper levels. Eg:

```
-->(- (+ 6 8) (+ 2 4))
```

8

We can also perform operations on other objects besides numbers. Suppose that we wanted to reverse the list (time flies like arrows). Trying the built in function reverse we get:

```
-->(reverse (time flies like arrows))
```

--- error in built in function [apply] ---

But the interpreter will be confused! It does not know that 'time' is data and not a function taking arguments 'flies', 'like' and 'arrows'. To indicate it is upset PC-LISP prints the error message above and alters the prompt. More on this later. What can we do to fix this? We must use the function 'quote' which returns its argu-

ments unevaluated, hence the name "quote".

```
-->(reverse (quote (time flies like arrows)))
(arrows like flies time)
```

Will give us the desired result (arrows like flies time). We can do the same thing without using the (quote) function directly. Remember the read macro ' above? Well it will replace the entry '(time flies like arrows) with (quote(time flies like arrows)). So more concisely we can ask PC-LISP to evaluate:

```
-->(reverse '(time flies like arrows))
(arrows like flies time)
```

This gives us the correct result without as much typing. You will now note that the subtraction of 2+4 from 6+8 could also have been entered as:

```
-->(- (+ '6 '8) (+ '2 '4))
```

8

However, the extra 's are redundant because a fixnum evaluates to itself. In general a LISP expression is evaluated by first evaluating each of its arguments, and then applying the function to the arguments, where the function is the first thing in the list. Remember that evaluation of the function (quote s1) returns s1 unevaluated. LISP will also allow the function name to be replaced by a function body called a lambda expression. Which is just a function body without a name. Example:

```
-->((lambda(x)(+ x 10)) 14)
```

24

Which would be processed as follows. First the parameters to the lambda expression are evaluated. That's just 14. Next the body of the lambda expression is evaluated but with the value 14 bound to the formal parameter given in the lambda expression. So the body evaluated is (+ x 10) where x is bound to 14. The result is just 24. Note that lambda expressions can be passed as parameters as can built in functions or user defined functions. Hence I can evaluate the following input. Note I use the] character to close the three open lists rather than typing))) at the end of the line.

```
-->((lambda(f x)(f (car x))) '(lambda(l)(car l)) '((hi]
```

hi

Which evaluates as follows. The parameters to the call which are the expressions '(lambda(l)(cdr l)) and '((hi)) are evaluated. This results in the expressions being returned because they are quoted. These are then bound to 'f and 'x respectively and the body of the first lambda expression is evaluated. This means that the expression ((lambda(l)(car l))(car ((hi)))) is evaluated. So again the parameters to the function are evaluated. Since the only parameter is (car ((hi))) it is evaluated resulting in (hi). This is then bound to l and (car l) is evaluated giving hi.

PC-LISP is also capable of handling all other function body kinds. These are lambda, nlambda, lexpr and macro kinds. These expression kinds may all have multiple bodies which are evaluated in order, the last one producing the value that is returned. See the section on BUILT IN FUNCTIONS and MACROS for more details on these kinds and how they operate. Better yet read LISPcraft.

16. Termination Of Expression Evaluation

There are three distinct ways that evaluation can terminate. First, evaluation can end naturally when there is no more work to do. In this case the resulting S-expression is printed on the

```
console and you are presented with the prompt "-->".  Second, you
```

can request premature termination by hitting the CONTROL-BREAK or CONTROL-C keys simultaneously (MS-DOS) or the INTR key (UNIX) (hereafter referred to as CONTROL-BREAK for both UNIX and MS-DOS). Note that this will only interrupt list evaluation, it will NOT interrupt garbage collection which continues to

completion. So, if you hit CONTROL-BREAK (ie INTR,CONTROL-C or CONTROL-BREAK) and you don't get any response, wait a second or two because it will respond after garbage collection ends. Finally, execution can terminate when PC-LISP detects a bad parameter to a built in function, a stack overflows, a division by zero is attempted, or an atom is unbound etc. In all cases but a normal termination you will be returned to a break error level. This is when the prompt

```
looks like 'er>'. This means that variable bindings are being
```

held for you to examine. So if the evaluation aborts with the message "error in built in function [car]", you can examine the atom bindings that were in effect when this error occurred by typing the name of the atom desired. This causes its binding to be displayed. When you are finished with the break level just hit CONTROL-Z plus ENTER (MS-DOS) or CONTROL-D (UNIX) and you will be placed back in the normal top level and all bindings that were non global will be gone. Note you can do anything at the break level that you can do at the top level. If further errors occur you will stay in the break level and any bindings at the time of the second error will be in effect as well as any bindings that were in effect at the previous break level. If bindings effecting atoms whose values are being held in the first break level are rebound at the second break level these first bindings will be hidden by the secondary bindings.

An error in built in functions 'eval' or 'apply' can mean two things. First, your expression could contain a bad direct call to eval or apply. Or, your code may be trying to apply a function that does not exist to a list of parameters, or trying to apply a bad lambda form. The interpreter does not distinguish an error made in a direct call by you to eval/apply or an indirect call to eval/apply, made by the interpreter on your behalf to get the expression evaluated.

There are a variety of math errors that are detected under certain implementations of PC-LISP. The MS-DOS and AT&T UNIX versions will both trap domain, argument singularity etc. errors as per the MATH(3M) library. These errors generate similar messages as the "error evaluating built in function" errors. The Berkeley UNIX math library will not trap these in the same way. Instead, you will get a system error message as descrbed by perror() in the UNIX programmers guide. You will have to look at the (showstack) to figure out which expression generated the error. The same is true for floating point exceptions and any other detectable system error such as (but not limited to) I/O errors. This is because PC-LISP checks for system errors after every evaluation so system errors such as "diskfull" will not pass unnoticed.

It is also useful to know what the circumstances of the failure were. You can display the last 20 evaluations with the command (showstack). This will print the stack from the top to the 20th element of the stack. This gives you the path of evaluation that lead to the error. For more information on the (showstack) command look in the section FUNCTIONS WITH SIDE EFFECTS OR THAT ARE EFFECTED BY SYSTEM.

It is possible but hopefully pretty unlikely that the interpreter will stop on an internal error. If this happens try to duplicate it and let me know so I can fix it.

17. Data Types In Pc-Lisp

PC-LISP has the following data types, 32 bit integers, double precision floating point numbers, lists, ports for file I/O, alpha atoms, strings, hunks, and MacLisp style arrays. The (type) function returns these atoms:

fixnum - a 32 bit integer (possibly 64 on some UNIXes)

flonum - a double precision floating point number.

list - a list of cons cells.

symbol - an alpha atom, with print name up to 254 chars which may include spaces tabs etc, but which should not include an (ascii 0) character. Symbols may have property, bindings and functions associated with them. Symbols with same print name are the same object.

string - A string of characters up to 254 in length. It has nothing else associated with it. Strings with same print name are not necessarily the same object.

port - A stream that is open for read or write. This type can only be created by (fileopen).

hunk - An array of 1 to 126 elements. The elements may be of any other type including hunks. Franz allows 127, the missing element is due to a space saving decision. This type can only be created by a call to (hunk) or (makhunk).

array - An array of any number of dimensions that can have any type of element. Size is restricted only by available memory. (no 64K limit)

Fixnums and flonums are together known as numbers. The read function will always read a number as a flonum and then see if it can represent it as a fixnum without loss of precision. Hence if the number 50000000000 is entered it will be represented as a flonum because it exceeds the precision of a fixnum. If a number has a decimal point or exponent specifier 'e' or 'E' in it, it is assumed to be a flonum even if there are no non zero digits following the radix point.

Fixnums and flonums will not appear the same when printed. The print function will output a flonum with a radix point and perhaps an exponent specifier if it will make the output smaller. Naturally, a fixnum never has a radix point.

Hunks when printed appear as { e0 e1 e2 .... eN }. They are indexed from zero. They cannot be entered, ie there is no read mechanism for creating them you must create them with a function call. Hunks are subject to compaction and relocation like any other PC-LISP object. The storage for the hunk itself comes from the heap, storage for the cell that handles the hunk comes from the cons, etc. space.

Arrays are implemented as 126-ary trees of hunks. They are also indexed from 0. Because they are implemented in terms of hunks, they are subject to compaction and reclaimation. The storage for the array is thus not really contiguous. However it appears so to the caller. Although you do not need to know how an array is implemented to use them, here is how it works in PC-LISP for your interest. Formally, an array tree is defined recursively as follows:

BASE : If the size of the array is < 126 the array tree is just a hunk the exact size as the array.

INDUCTION: If the size of the array is >= 126, the array tree is a hunk of size exactly 126 or 125. The entries 0 .. 124 contain array trees each of which has size equal to the parent's size divided by 125 (truncated division). If the remainder of the size of the array divided by 125 is zero, the hunk is of size 125 and has no 125th entry. If the remainder of the size of the array divided by 125 is non zero, then the size of the hunk is 126 and the 125th entry is used to either store the remainder array, or the remainder element as follows. If the remainder array is of size exactly 1, it is not stored, the 125th entry of the parent is used to hold the entry instead. If however the remainder is greater than 1, the 125 entry of the parent holds a hunk of size equal to the remainder.

Arrays when printed will print as array[nnn] where nnn is the number of elements in the array. Multidimensional arrays are stored in exactly the same way as linear arrays. The only difference is in how the element number is computed when doing array accesses. They will also print as array[nnn] where nnn is the total number of elements in all dimensions of the array. It is possible to allocate some pretty big arrays in PC-LISP, however you will need to adjust the LISP_MEM environment variable H option to make sure there is enough heap space for them.

Also note that the array hunk tree is allocated all at once so for large arrays it takes some time to initialize. Also, the array access functions (store) and (arraycall) are provided as macros in pc-lisp.l. Finally note that unlike Franz, you cannot specify a user written access function for the array or alter any of the other array specific data besides the raw array tree.

18. The Built In Functions And Variables

Following is a list of each built in function. I will denote the allowed arguments as follows:

- a1...aN are alpha atom parameters, type symbol.
- h1...hN are string or alpha atoms, type string or symbol.
- x1...xN are integer atom parameters, type fixnum (32bits).
- f1...fN are double precision reals, type flonum.
- n1...nN are number atom parameters, type flonum or fixnum.
- z1...zN are numbers but all are of the same type.
- l1...lN are lists, must be nil or of type list.
- p1...pN are port atom parameters, type port.

- s1...sN are S-expressions (any atom type or list)

- H is a hunk.

- A is a symbol which is bound to an array.

Additional Definitions: ~~~~~~~~~~~~~~~~~~~~ "{a|d}+" means any sequence of characters of length greater than 0 consisting of a's and d's in any combination. This defines the car,cdr,cadr,caar,cadar... function class as follows: "c{a|d}+r".

"[-stuff-]" indicates that -stuff- is/are optional and if not provided a default will be provided for you.

"*-stuff-*" indicates that -stuff- is not evaluated. An example of this is the function (quote *s1*) whose single S- expression parameter s1 is enclosed in *'s to indicate that quote is passed the argument s1 unevaluated.

For the simpler functions I will describe the functions using a sort of "if (condition) result1 else result2" notation which should be pretty obvious to most people. For functions that are a little more complex I will give a short English description

```
and perhaps an example. If the example code shows the '-->'
```

prompt you should be able to type exactly what follows each prompt and get the same responses from PC-LISP. If the example

```
does not show a '-->' prompt the example is a code fragment and
```

will not necessarily produce the results shown.

19. Predefined Global Variables (Atoms)

A number of atoms are globally prebound by PC-LISP. These variables are testable and setable by you but in some cases altering the bindings is highly inadvisable. Note that a binding can be inadvertantly altered by defining one of these atoms as a local or parameter atom to a function or a prog, or directly by using 'set' or 'setq'.

"displace-macros" - This atom when non nil will cause macro expansion to be follwed by code substitution if such substitution is possible. The default value is nil meaning no substitution.

"t" - This atom means 'true', it is bound to itself. Various predicates return this to indicate a true condition. You should NOT change the binding of this atom, to do so will cause PC-LISP to produce incorrect answers.

"nil" - This is not really an atom, it represents the empty list (). It is not bound to () but is rather equivalent to () in all contexts. Any attempt to create a symbol with print name "nil" will result in ().

"$ldprint" - Is initially bound to "t". When not bound to "nil" this atom causes the printing of the -- [file loaded] -- message when the function (load file) is executed. When "nil" this atom prevents the printing of the above message. This is useful when you want to load files silently under program control. It will also inhibit the pc-lisp.l loaded message.

"$gcprint" - Is initially bound to "nil". When bound to "nil" garbage collection proceeds silently. If bound non "nil" then at the end of a garbage collection cycle 4 numbers are printed. The first is the number of collection cycles that have occured since PC-LISP was started, the second is the percentage of cons cells that are in use, the third the percentage of alpha cells, and the third the percentage of heap space that is in use. These last three numbers are exactly what you get back with a call to (memstat).

"$gccount$ - Is initially bound to 0. It increases by one every time garbage collection occurs. This number is the same as the first number printed when $gcprint is bound non "nil" and garbage collection occurs. While you can set $gccount$ to any value you want, its global binding will be reset to the correct garbage collection cycle count whenever collection finishes.

"piport", "poport", "errport" - Are bound to the standard input, standard output and standard error ports respectively. You can use these to force patom, princ, print and pp-form to send their output to the standard output or error. Or, to force read and readc to get their input from the standard input. They are initially bound to the keyboard and screen. You can alter their bindings if you wish but this is not recommended.

20. The Math Functions

Functions that operate on numbers, fixnums or flonums. Note

```
that the arrow --X--> may indicate what type is returned. If X is
```

's' then the same type as the parameter(s) selected is returned. If X is 'f' then a flonum type is returned. If X is 'x' then a fixnum is returned. If X is 'b' then the best type is returned, this means that a fixnum is returned if possible. Note that you should use fixnums together with "1+, 1- zerop" when ever possible because doing so gives nearly a 50% decrease in run time for many expressions, especially counted loops or recursion.

21. Trig And Other Math Functions

```
(abs  n1)     --s-> absolute value of n1 is returned.
(acos n1)     --f-> arc cosine of n1 is returned.
(asin n1)     --f-> arc sine of n1 is returned.
(atan n1 n2) --f-> arc tangent of (quotient n1 n2).
(cos n1)      --f-> cosine of n1, n1 is radians
(exp n1)      --f-> returns e to the power n1.
(expt n1 n2) - b-> n1^n2 via exp&log if n1 or n2 flonum.
(fact x1)     --x-> returns x1! ie x1*(x1-1)*(x1-2)*....1
(fix n1)      --x-> returns nearest fixnum to number n1.
(float n1)    --f-> returns nearest flonum to number n1.
(log n1)      --f-> natural logarithm of n1 (ie base e).
(log10 n1)    --f-> log base 10 of n1 {not present in Franz}
(lsh x1 x2)   --x-> x1 left shifted x2 bits (x2 may be < 0).
(max n1..nN) --s-> largest of n1...nN or (0 if N = 0)
(min n1..nN) --s-> smallest of n1..nN or (0 if N = 0)
(mod x1 x2)   --x-> remainder of x1 divided by x2.
(random [x1])--x-> random fixnum, or random in 0...x1-1.
(sin n1)      --f-> sine of n1, n1 is radians.
(sqrt n1)     --f-> square root of n1.
(1+ x1)       --x-> x1+1.
(add1 n1)     --b-> n1+1 (done with fixnums if n1 is fixnum).
(1- x1)       --x-> x1-1.
(sub1 n1)     --b-> n1-1 (done with fixnums if n1 is fixnum).
```

22. Basic Math Functions

```
(* x1 ...... ..xN) --x-> x1*x2*x3*.....nN (or 1 if N = 0)
(times n1 .. ..nN) --b-> n1*n2*n3......nN (or 1 if N = 0)
(product n1....nN) --b-> Ditto
(+ x1....... ..xN) --x-> x1+x2+x3+.....xN (or 0 if N = 0)
(add n1 .......nN) --b-> n1+n2+n3+.....nN (or 0 if N = 0)
(sum n1 .......nN) --b-> Ditto
(plus n1.......nN) --b-> Ditto
(- x1....... ..xN) --x-> x1-x2-x3-.....xN (or 0 if N = 0)
(diff n1.......nN) --b-> n1-n2-n3-.....nN (or 0 if N = 0)
(difference....nN) --b-> Ditto
(/ x1....... ..xN) --x-> x1/x2/x3/.....xN (or 1 if N = 0)
(quotient n1...nN) --b-> n1/n2/n3/.....xN (or 1 if N = 0)
```

Note that the Basic functions that operate on numbers will return a fixnum if the result can be stored in one.

23. The Boolean Functions

These functions all return boolean values. The objects t and nil represent true and false respectively. Note however that most functions treat a non nil value as being t. t is a predefined atom whose binding is t while nil is not a real atom but rather a lexical item that is EQUIVALENT to () in all contexts. Hence nil and () are legal as both an atom and a list in all functions.

Note when comparing flonums you cannot use (eq) because they are not identical objects. (eq) however will work on fixnums as in Franz.

```
(alphalessp h1 h2) ---> if (h1 ASCII before h2) t else nil;
(arrayp s1)        ---> if (s1 is type Array) t else nil;
(atom s1)          ---> if (s1 not type list) t else nil;
(and s1 s2 .. sN)  ---> if (a1...aN all != nil) t else nil;
(boundp a1)        ---> if (a1 bound) (a1.eval(a1)) else nil;
(eq s1 s2)         ---> if (s1,s2 same obj/fix) t else nil;
(equal s1 s2)      ---> if (s1 has s2's structure) t else nil;
(evenp n1)         ---> if (n1 mod 2 is zero) t else nil;
(fixp s1)          ---> if (s1 of type fixnum) t else nil;
(floatp s1)        ---> if (s1 of type flonum) t else nil;
(greaterp n1...nN) ---> if (n1>n2>n3...>nN) t else nil;
(hunkp s1)         ---> if (s1 of type hunk) t else nil;
(lessp n1...nN)    ---> if (n1<n2<n3...<nN) t else nil;
(listp s1)         ---> if (s1 of type list) t else nil;
(minusp n1)        ---> if (n1 < 0 or 0.0) t else nil;
(not s1)           ---> if (s1 != nil) nil else t;
(null s1)          ---> Ditto
(numberp s1)       ---> if (s1 is fix of float) t else nil;
(numbp s1)         ---> Ditto.
(or s1 s2 .. sN)   ---> if (any si != nil) t else nil;
(oddp n1)          ---> if (n1 mod2 is non zero) t else nil;
(plusp n1)         ---> if (n1 > 0 or 0.0) t else nil;
(portp s1)         ---> if (s1 of type port) t else nil;
(zerop n1)         ---> if (n1 = 0 or 0.0) t else nil;
(< z1 z2)          ---> if (z1 < z2) t else nil;
(= z1 z2)          ---> if (z1 = z2) t else nil;
(> z1 z2)          ---> if (z1 > z2) t else nil;
```

Note carefully the difference between (eq) and (equal). One checks for identical objects or fixnums, ie the same object, while the other checks for two objects that have the same structure and identical leaves.

Note that the (and) and (or) functions evaluate their arguments one by one until the result is known. Ie, short circuit evaluation is performed.

Note that proper choice of fixnums over flonums and proper choice of fixnum functions can yield large performance improvements.

24. List & Atom Creators And Selectors

These functions will take lists and atoms as parameters and return larger or smaller lists or atoms. They have no side effects on the LISP system nor are their results affected by anything other than the values of the parameters given to them. These functions are all nondestructive as they do not alter their parameters in any way.

```
(append l1..ln)    ---> list made by joining all of l1..ln.
```

If any of l1..ln is nil they are ignored.

```
(ascii n1)         ---> atom with name 'char' where 'char'
```

has ordinal value n1:(0 < n1 < 256).

```
(assoc s1 s2)       ---> if s2 is a list of (key.value) pairs
                         then assoc --> (key.value) from s2,
```

where (equal key s1) is t else nil.

```
(car l1)            ---> first element in l1. If l1 is nil
```

car returns nil.

```
(cdr l1)            ---> Everything but the car of l1. If
```

l1 is nil cdr returns nil.

```
(c{a|d}+r l1)       ---> performs repeated car or cdr's on
```

l1 as given by reverse of {a|d}+. Returns nil if it cars or cdrs off the end of a list.

(character-index h1 h2) -x-> Returns the index (from 1) of first char in h2 in h1. h2 can be a fixnum ascii value. Returns nil if none.

```
(concat s1 .. sN)   ---> Forms a new atom by concatenating
```

all the strings,atoms,fixnums and flonums print names.

```
(cons s1 s2)        ---> list with s1 as 1st elem s2 is rest.
```

If s2 is nil the list has one element. If s2 is an atom the pair print with a dot. (cons 'a 'b) will print as (a . b).

```
(explode h1)        ---> list of chars in print name of h1.
```

If h1 is nil returns (n i l)

```
(exploden h1)       ---> list of ascii values of chars in h1.
```

If h1 is nil returns (110 105 108).

```
(get_pname h1)      ---> String equal to print name of atom
```

h1 or same as string h1.

```
(hunk-to-list H)    ---> Returns a list whose elements are
                         (eq) to those of hunk H and in the
```

same order.

```
(implode l1)        ---> atom with name formed by compressing
```

first char of each atoms print name in l1. Imploding (n i l) returns the empty list nil. Small fixnums in 0..255 are treated as ascii chars.

```
(last  l1)          ---> returns the last element in l1.  If
```

l1 is nil it returns nil.

```
(length l1)         -x-> fixnum = to length of list l1.
```

The length of nil is 0.

```
(listarray A [n1]) ---> Returns all of A or just first n1
```

elements as a list.

```
(list s1 s2...sN)   ---> a list with elements (s1 s2 ...sN)
```

If N = 0 list returns nil.

```
(member s1 l1)      ---> If (s1 (equal) to element of l1)
```

returns l1 (from match) else nil.

```
(memq  s1  l1)      ---> If (s1 (eq) to element of l1)
```

returns l1 (from match) else nil.

```
(nth n1 l1)         ---> n1'th element of l1 (indexed from 0)
```

like (cad...dr l1) with n1 d's.

```
(nthcdr n1 l1)      ---> returns result of cdr'ing down the
```

list n1 times. If n1 < 0 it returns

```
                         (nil l1).

(nthchar h1 n1)     ---> n1'th char in the print name of h1
```

indexed from 1.

```
(pairlis l1 l2 l3) ---> l1 is list of atoms. l2 is a list
```

of S-expressions. l3 is a list of

```
                         ((a1.s1)....) The result is the
```

pairing of atoms in l1 with values in l2 with l3 appended (see assoc).

```
(quote *s1*)        ---> s1, unevaluated!

(reverse l1)        ---> copy of l1 reversed at top level.

(type s1)           ---> list,flonum,port,symbol, fixnum,
```

hunk or array as determined by the type of the parameter s1.

(sizeof h1)

Will return the number of bytes necessary to store an object of type h1. Legal values for h1 are 'list,'symbol,'flonum, 'fixnum, 'string , 'hunk, 'array and 'port. The size returned is the amount of memory used to store the cell, incidental heap space, property list space, binding stack space and function body space is not counted for types 'symbol, 'string, 'hunk or 'array.

(stringp s1)

Will return t if the S-expression s1 is of type string, otherwise it returns nil.

(substring h1 n1 [n2])

If n1 is positive substring will return the substring in string h1 starting at position n1 (indexed from 1) for n2 characters or until the end of the string if n2 is not present. If n1 is negative the substring starts at |n1| chars from the end of the string and continues for n2 characters or to the end of the string if n2 is not present. If the range specified is not contained within the bounds of the string, nil is returned.

(memusage s1)

Will return the approximate amount of storage that the S- expression s1 is occupying in bytes. The print-name heap space is included in this computation as are file true name atoms. This function is not smart, it will count an atom twice if it is found more than once in the list. The space count does not include storage needed for binding stacks, property lists, or function bodies that are associated with a particular atom. Hunk and string space include the heap space owned by the cell. If an S-expression is a list all the elements (memusage) will be added to get the total (memusage) for the list.

25. Noninterning/Interning Functions

Unless otherwise stated in this manual, any function that returns an atom will intern it (put it on the oblist). However the following functions are not included in the above statement. Note also that the list returned by (oblist) is a copy of the real oblist. Note carefully that the atoms created by read are interned. See a really good LISP

manual on this stuff because it can be really confusing.

(copysymbol a1 s1)

Returns an UNINTERNED copy of atom a1. If the flag parameter s1 is non nil then the returned atom has property, value, and function definitions eq to a1 otherwise its property, value and function definitions are nil, undefined, and undefined respectively.

(gensym [a1])

Returns an UNINTERNED atom whose print name is of the form Xnnnnn where X is either 'g' or the print name of a1 (if a1 is provided) and nnnnnn is some number such that no interned or uninterned atom in the system has the same print name. Note that the the existence of a clashing interned or uninterned atom is checked before selecting the value of nnnnn.

(intern a1)

Will INTERN a1 on the oblist. If an atom with the same print name as a1 is already on the oblist the EXISTING interned atom is returned. Otherwise, a1 is physically added to the oblist and is returned.

(remob a1)

Will return a1 after having physically removed a1 from the oblist. Future calls to read will create a new atom with the same print name as a1. This can be confusing if a1 had a function definition, property, or value associated with it.

(maknam l1)

Takes a list of symbols/strings/fixnums as parameter and returns an UNINTERNED atom whose print name is the concatenation of the first characters in the print names of every symbol and/or the ascii characters whose values are given as fixnums in l1.

(uconcat a1 a2 ... aN)

Returns an UNINTERNED atom whose print name is the concatenation of each of the print names of a1...aN. If N=0, or if N=1 and a1 is nil, then the empty list nil is returned. Note that the empty list nil is neither interned or uninterned because it is not really an atom. Like concat, it handles flo/fixnums.

26. File I/O Functions

These functions manipulate port atoms and allow character or S-expression I/O. A port atom is returned by (fileopen) and will print as %file@nn% where 'file' is the name of the port and nn is the file number or -1 if the file is closed. All I/O is checked and an error closing, reading or writing a port will be trapped.

(close p1)

Closes the port p1 and returns t. It will then invalidate the port p1. Any further I/O to p1 is illegal. I/O errors may be trapped when the close is issued.

(fileopen h1 h2)

Opens a file whose name is h1 for mode h2 access. h1 should be a path or device name. h2 should be one of 'r,'w,'a,'r+,'w+, or 'a+ meaning respectively: (r) Read only. (w) Truncate or create for writing. (a) Open for writing at end of file or create for writing. (r+) Open for update (read+write). (w+) Truncate or create for update. And (a+) open or create for update at end of file. MS-DOS device names like 'con', 'lpt1' etc are accepted. Fileopen will not search for a file on the PATH or in the LISP_LIBs. The MICROSOFT C MS-DOS version allows the addition of a mode 'b meaning binary (no newline translation). It is appended to the above modes eg 'rb or 'wb meaning read binary, write binary etc. Depending on the compiler/operating system there may be other modes allowed. (See the LibC manual for fopen(3S) mode strings). Fileopen returns an open port atom, or nil if the file/device could not be opened in the requested I/O mode.

(filepos p1 [x1])

If fixnum parameter x1 is not provided filepos will return the current file position where the next read/write operation will take place for port p1. If x1 is provided it is interpreted as a new position where the next read/write should take place. The read/write pointer is seeked accordingly and the value x1 is returned if the seek completes successfully. Otherwise nil.

(load h1)

Will try to find the file whose name is h1 and load it into PC-LISP. Loading means reading every list, and evaluating it. The results of the evaluation are NOT printed on the console. In trying to find the file h1, load uses the following strategy. First it looks for file h1 in the current directory, then it looks for h1.l in the current directory. Then it gets the value of the environment variable LISP_LIB which should be a comma separated sequence of MS-DOS paths (exactly the same syntax as for PATH). It then repeats the above searching strategy for every directory in the path list. For example if I entered this from the COMMAND shell:

"set LISP_LIB=c:\usr\libs\lisp\bootup;c:\lisp\work\;"

then ran PC-LISP, it would try to load the file PC-LISP.L first from the current directory, then from the two directories on the C drive that are specified in the above assignment. Future calls to (load h1) will also look for files in the same way. When a file has been successfully loaded PC-LISP examines the value of atom $ldprint. If this value is non-nil (default is t) PC-LISP will print a message saying that the file was loaded successfully. If this value is nil then no message is printed. In either case if the load is successful a value of t is returned and if the load fails a value of nil is returned.

```
(patom s1 [p1]) & (princ s1 [p1])
```

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ Both cause the S-expression s1 to be printed without delimiters or escapes on the output port p1, or on the standard output if no p1 parameter is given. Without delimiters means that if an atom has a print name that is not legal without the | | delimiters or without an escape \, neither will be added when printing the atom with patom. Patom returns s1 while princ returns t. Strings will print without quotes or escapes.

(print s1 [p1])

Will cause the S-expression s1 to be printed with delimiters and escapes if necessary on the output port p1, or on the standard output if no p1 parameter is given. All atoms that would require | | delimiting, strings that require " " delimiting and characters that would have to be preceeded by the escape to be input, will be printed with the delimiters and any necessary escapes. If a character is one of the format characters tab, back space, carriage return, line feed or form feed, it will print preceeded by the escape as \t \b \r \n or \f respectively. If the characters ascii value is < 32 or > 126 and it is not a format character, it will print as \?. Print returns the expression s1.

(read [p1 [s1]])

Reads the next S-expression from p1 or from the standard input if p1 is not given and returns it. If s1 is given and end of file is read the read function will return s1. If s1 is not given and end of file is read the read function will return nil.

(readc [p1 [s1]])

Reads the next character from p1 or from the standard input if p1 is not given and returns it as an atom with a single character name. If s1 is given and end of file is read the readc function will return s1. If s1 is not given and end of file is read the readc function will return nil.

(resetio)

Will close all open files except the standard input, standard output and standard error ports. It is useful when too many (load)s are aborted due to errors in the load file. It always returns true.

(sys:unlink h1)

Will erase the file whose name is the print name of atom h1. If the erase is successful a value of 0 is returned. If the erase is unsuccessful a value of -1 is returned.

(truename p1)

Will return an atom whose print name is the same as the name of the file associated with port p1. This is just the same as the value printed between the % and @ signs when a port is printed.

(flatsize s1 [x1])

Returns the number of character positions necessary to print s1 using the call (print s1). If x1 is present then flatsize will stop computing the output size of s1 as soon as it determines that the size is larger than x1. This feature is useful if you want to see if something will fit in some small given amount of space but not knowing if the
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

list is very big or not.

(flatc s1 [x1])

Returns the number of character positions necessary to print s1 using the call (patom s1). x1 is the same as in flatsize.

(pp-form s1 [p1 [x1]])

Causes the expression s1 to be pretty-printed on port p1 indented by x1 spaces. If p1 is absent the standard output is assumed. If x1 is absent an indent of 0 is assumed. If s1 contains a list such as (prog .... label1 ... label2...) the normal indenting will be ignored for label1 & label2 etc. This causes the labels to stand out. For example IF the following function were present in PC-LISP then I could run pp-form:

```
-->(pp-form (getd 'character-index-written-in-lisp))
(lambda (a c)
        (prog (n)
              (setq n 1 a (explode a))
              (cond ((fixp c) (setq c (ascii c))))
nxt:
              (cond ((null a) (return nil)))
              (cond ((eq (car a) c) (return n)))
              (setq n (1+ n) a (cdr a))
              (go nxt:)))
```

(drain [p1])

Will cause p1 or the poport to be drained. If the port is an input port then all unread characters will be discarded. If the port is an output port then all unwritten characters will be flushed.

(zapline)

Will cause all characters up to and including the next new line (ascii 10) to be read and discarded. The port that is read is the last one that was used for input. This function is useful for defining comment skipping macros. For example.

```
-->(setsyntax '# 'vsplicing-macro
'(lambda()(zapline)))
```

Will define # as a comment starting character. When it is encountered by (read) it will call (zapline) which skips all characters up to an including the end of the line. Since (zapline) returns nil and the macro is 'vsplicing-macro, nil is spliced into the input list. In other words the nil has no effect on the input list. This is the reason for reading from the last file used for input.

27. Functions With Side Effects Or That Are Effected By System

These functions will either have an effect on the way the system behaves in the future or will give you a result about the way the system has behaved in the past. Future calls will not necessarily give the same results.

(def *a1* *l1*)

a1 is a function name and l1 is a lambda, nlambda, lexpr or macro body. The body is associated with the atom a1 from now on and can be used as a user defined function. Def returns a1. Note that a lambda expressions parameter list may contain the &aux,&optional and &rest flags. See Defun for details.

```
-->(def first  (lambda(x)(car x)))
-->(def llast  (lexpr(n)(last (arg n))))
-->(def myadd  (nlambda(l)(eval(cons '+ l))))
-->(def firstm (macro(l)(cons 'car (cdr l))))
-->(def X*2orY (lambda(x &optional(y 2))(* x y)))
```

(defun *a1* [*a2*] *s0* *s1* *s2* ....*sN*)

Defun will do the same job as "def" except that it will build the expression body for you. a1 is the name of the expression that you are defining, a2 is an optional expression kind indicator which may be either expr, fexpr or macro. The default is expr. These kinds correspond directly to lambda, nlambda and macro forms. s0 specifies the formal parameters to the expression. Usually this is just a list of symbols. If it is a single symbol it is assumed that the symbol is the single parameter to an lexpr form and an lexpr form will be constructed from the ensuing bodies s1....sn. If it is a list of symbols then a lambda, nlambda or macro body will be constructed from the bodies s1...sn according to the kind specified by parameter a2. For example, these calls to defun do the same job as the above calls to def.

```
-->(defun first(x)(car x))
-->(defun llast n (last (arg n)))
-->(defun myadd fexpr(l)(eval(cons '+ l)))
-->(defun firstm macro(l)(cons 'car (cdr l)))
-->(defun X*2orY (x &optional(y 2)) (* x y))
```

A simple function definition (ie not an fexpr or macro) may contain the flags &optional,&rest and &aux in its parameter list. These flags must occur in the above order if all are present. They have the following effects: The function will be allowed to take a variable number of args (via an lexpr) and the parameters up to the &optional flag must be present when the function is called. The parameters after the &optional do not have to be present, and if not present they will default to nil unless the formal parameter form (var default) ie (y 2) is used. If this is the case the parameter will be bound instead to 'default'.

If &rest is present it may be followed by exactly one parameter name. When the function is called any unaccounted for 'extra' parameters will be turned into a list and bound to this parameter. If &aux is present then the symbols present in the following forms will all be bound either to nil, or if the form is (var defualt) the symbol 'var' will be bound initially to 'default'. This has the effect of introducting local variables with either nil or predefined default values. After this nasty English description, I think that an example is in order.

```
-->(defun foo(a &optiona(b 2) c &rest d &aux (e 2) f)
             (list a b c d e f))
```

foo

```
-->(foo 1)
(1 2 nil nil 2 nil)      ; a=1 b=2 c=nil d=nil e=2 f=nil
-->(foo "hi" "there")
("hi" "there" nil nil 2 nil)    ; b="there" not the default.
-->(foo 1 2 3 4)
(1 2 3 (4) 2 nil)               ; e = unaccounted for parms.
-->(foo)
```

--- error evaluating built in function[arg] ---

```
er>(pp foo)
(def foo (lexpr(_N_)
         ((lambda(a b c d e f)(list a b c d e f))
          (arg 1)
          (arg? 2 2)
          (arg? 3 nil)
          (listify 4)
```

2 nil)))

This function has 1 required argument 'a', 2 optional arguments 'b' and 'c' whose default values are 2 and nil respectively. Any additional arguments are to be bound as a list to 'e'. The function has two local variables 'e' and 'f' whose default values are respectively 2 and nil. The function when called simply makes a list of all it's arguments, any left over arguments, and its local variables. Invoking the function with a varying number of arguments shows the effect in the returned list. The (foo) invokation shows that at least one argument is required

otherwise the (arg 1) function fails. Finally I dumped the actual function definition. It is an lexpr expression with an immediate invokation of a lambda expression with arguments ((arg 1).....nil). The special function (arg? nn exp) is like (arg nn) except that if nn is not in the range of the actual parameters it returns exp. Franz does not do this but PC-LISP does because it is much more effecient than using a (cond) to get the arg or default value. Because the (arg?) function is not present in Franz, I do not recommend you use it directly, instead use defun to create the correct forms for you, this will be portable.

(exec *s1* *s2* .... *sN*)

Will execute the program s1+' '+s2+' '+....+' '+sN. This is done by using the system() call. For example if you are in PC- LISP and you want to edit a file you could type.

```
-->(exec zed "lisp.h")
```

And the command "zed lisp.h" would be executed. Note that I put quotes around the lisp.h file name because the '.' could cause syntax problems. (exec) will return the return status of the executed command as a fixnum. Note that if you get -1 back from (exec) either the command cannot be found, or there is not enough memory to run it. If there is not enough memory to run the command you must set your LISP_MEM B setting a little smaller to leave some memory for the commands you wish to execute. Note that you can start up an MS-DOS shell as follows:

```
-->(exec command)
```

Which will execute command.com (assuming it is on your PATH) and put you in the shell. You can then execute normal DOS commands and return to PC-LISP by typing exit at the DOS prompt. The pc-lisp.l file contains a definition of (shell) which does exactly this. Note for UNIX you would have to do something like:

```
-->(exec "/bin/sh") or (exec "/bin/csh") etc...
```

(exit)

PC-LISP will exit to whatever program envoked it this is usally the COMMAND.COM program. Depending on how big you set LISP_MEM MSDOS may ask for a system disk to reload COMMAND.COM. Note that the video mode will be left alone if you call exit. But if you leave via CONTROL-Z the video mode will be reset to 80x25B&W if you have changed it via (#scrmde#)).

(gc)

Starts garbage collection of alpha and cell space. Returns t when the cycle has ended. Reducing the settings of the LISP_MEM blocks(B) or alpha(A) and or increasing the value of heap(H) will decrease the time needed to complete garbage collection but will reduce the available cell memory thus increasing the number of garbage collections that are required in a given period. (gc) is a useful way to spend idle time. If for example you have displayed some computation and are waiting for a response from the user, you can invoke (gc) after prompting but before reading the response. Invoking (gc) yourself reduces the number of times PC-LISP must do it for you.

(get a1 a2)

Will return the value associated with property key a2 in a1's property list. This value will have been set by a previous call to (putprop a1 s1 a2). Example:

```
-->(get 'frank 'lastname)
```

(getd a1)

Will return the array, lambda, nlambda, fexpr or macro expression that is associated with a1 or nil if no such expression is associated with a1.

(getenv h1)

Will return an atom whose print name is the string set by environment variable h1. For example we can get the PATH variable setting by evaluating (getenv 'PATH). Note that these must be in upper case because MS-DOS

converts the variable names to upper.

(hashtabstat)

Will return a list containing 503 fixnums. Each of these represents the number of elements in the bucket for that hash location in the heap hash table. 503 is the size of the hash table. This is not especially useful for you but it gives me a way of checking how the hashing function is distributing the heap using cells. Heap using cells are symbol, string and hunk. The cell itself is allocated from the alpha or other memory blocks while its variable length space is allocated from the heap. Hence this table contains the oblist plus strings and hunks and uninterned symbols. This table should not be confused with the oblist which runs through this table.

(memstat)

Returns three fixnums. The first is the percentage of cell space that is in use. The second is the percentage of alpha cell space and the third is the percentage of heap space in use. When any of these reach 100%, garbage collection will occur. Alpha and cell space is collected together. Heap space is only collected when you run out. After garbage collection you will see these three percentages drop. The alpha and cell percentages should drop to tell you how much memory is actually in use at that moment. The heap space when compacted and gathered will not necessarily drop to indicate how much you really have left. This is because heap space is gathered in blocks of 16K, not all at once as with atoms and cells. So, there will almost certainly be more than 20% free heap space in other non compacted blocks even if memstat reports 80% of the heap space is in use.

(oblist)

Returns a list of most known symbols in the system at the current moment. Note that if you call oblist and assign the result somewhere you will cause every one of those objects to be kept by the system. If there are lots of large alpha atoms the heap and alpha space will be tied up until you set the assigned variable to some other value. Several special internal atoms are not placed in the returned list to keep them out of user code.

(plist a1)

Will return the property list for atom a1. The property list is of the form ((ke1 . value1)(key2 . value2)...(keyn . valuen)). Note that plist returns a top level copy of the property list because remprop destroys this list's top level structure.

(putd a1 l1)

Identical to "def" except that the parameters a1 and l1 are evaluated. This allows you to write functions that create other functions and then add them to the LISP interpreter.

(putprop a1 s1 a2)

Adds to the property list of a1 the value s1 associated with the property indicator a2. It returns the value of a1. For example: (putprop 'Peter 'AshwoodSmith 'LastName)

(remprop a1 a2)

Removes the property associated with key a2 from the property list of atom a1. The top level structure of the property list is actually destroyed. It returns the old property list starting at the point where the deletion was made.

(set a1 s1)

Will bind a1 to s1 at current scope level or globally if no scope yet exists for a1. (set) returns s1.

(setplist a1 l1)

Will set the property list of atom a1 to the list l1 where the list must be ((keyn.valn)..). It returns this new list l1.

(setq *a1* s1 *a2* s2 ..... *an* sn)

Like set but takes any number of atoms "an" and values "sn". (setq) evaluates only the values s1...sn,(not the atoms) and then binds the values to the atom as per (set). If the atom is unbound before the call to setq it will be bound globally otherwise only the current binding is altered. The expressions are evaluated left to right and the bind is made after each expression is evaluted. Setq will return the value of the last evaluated expression. If no

parameters are given (setq) returns nil.

```
-->(setq x '(a b c)
(a b c)
-->(setq x (cdr x) y (car x))
```

b

```
-->x
(b c)
-->y
```

b

(PAR-setq *a1* s1 *a2* s2 ..... *an* sn)

PAR-setq does not exist in Franz Lisp. It was added to PC- LISP to help with the implementation of a (do) macro. PAR-setq does the same thing as (setq) except that the assignments are done in parallel. Ie the bindings are only done after all of the s1 to sn expressions have been evaluated. PAR-setq should not be used unless portability is not important.

```
-->(PAR-setq x '(a b c))
(a b c)
-->(PAR-setq x (cdr x) y (car x))
```

a

```
-->x
(b c)
-->y
```

a

(setsyntax a1 a2 l1)

Is a way of defining a read expression macro l1 to be associated with chracter a1 and invoked in 'vmacro or 'vsplicing- macro mode depending on a2. This function allows you to alter the way that (read) works. Basically after calling setsyntax the expression l1 will be invoked whenever the character a1 is found in the input stream and this character is not escaped or hidden in a comment or delimiters of some kind. For example a macro : that pretty prints the following function name could be defined as follows:

```
-->(setsyntax '|:| 'vmacro '(lambda()(list 'pp (read))))
```

Then if I typed :pp at the input prompt the character : would be read causing the expression (list 'pp (read)) to be invoked. This would then read the pp atom and construct the list (pp pp) which would then be passed back to the read function which would pass it back to the eval loop which will evaluate it and pretty print the function pp. Read macro expressions are lambda expressions that take no parameters. Any calls to (read) must not have any arguments, (read) will know where to read the next expression from because of a global binding performed by the read macro driver on behalf of the read function.

Splicing macros are also available. Just replace the 'vmacro parameter with 'vsplicing-macro. What will happen is that the returned list will be spliced into the input expression, rather than forming a sublist expression in the current input. This is useful if you want to define your own comment delimiters and return nil. For example let's define a new comment delimiter say the < and > characters.

```
-->(defun SkipToEnd()
          (cond ((eq (readc) '|>|) nil)
                (t (SkipToEnd))))
```

SkipToEnd

```
-->(setsyntax '|<| 'vsplicing-macro '(lambda()(SkipToEnd)))
```

t

```
-->(and t <junk junk junk> t)
```

t

What I have done is first write a comment skipping function that just reads input character by character until the > is found. I then associated the character '<' with a lambda expression that calls this skipper. The macro is a splicing macro as (and t <junk...junk> t) demonstrates. Think about what would have happened if the macro were non splicing and I put a comment in the (and ....) list. Try it and see, then you will know why splicing macros are needed.

```
(sys:time) & (time-string [n1])
```

~~~~~~~~~~~~~~~~~~~~~~~~~~ sys:time returns a fixnum representing the time in seconds since UNIX/MS-DOS creation. Time-string takes a fixnum n1 and returns to a human readable string representation of the time n1. If n1 is not provided time-string uses the current sys:time.

(trace [*a1* *a2* *a3* ..... *an*])

Will turn on tracing of the user defined functions a1...an. Note that you cannot trace built in functions. If you call trace with no parameters it will return a list of all user defined functions that have been set for tracing by a previous call to trace, otherwise trace returns exactly the list (a1 a2...an) after enabling tracing of each of these user defined functions. If any of the atoms is not a user defined function trace stops and returns an error. All atoms up to the point of error will be traced.

(untrace [*a1* *a2* *a3* ..... *an*])

Will disable tracing of the listed functions which must all be user defined. If no parameters are given it disables tracing of all functions. Untrace returns a list of all functions whose tracing has been disabled. Here is a demonstration of how you can use them. This is the sort of sequence that you should see on the console. The comments ;... were added to tell you what is going on.

```
-->(defun factorial(n)                  ; define n! = n * (n-1)!
          (cond ((zerop n) 1)
                (t (* n (factorial (1- n]
```

factorial

```
-->(trace factorial)                    ; ask LISP to trace n!
```

(factorial)

```
-->(factorial 5)                        ; ask LISP for 5!
<enter> factorial( 5 )                  ; entered with parm=5
 <enter> factorial( 4 )                 ;    "      "    "  4
  <enter> factorial( 3 )                ;    "      "    "  3
   <enter> factorial( 2 )               ;    "      "    "  2
    <enter> factorial( 1 )              ;    "      "    "  1
     <enter> factorial( 0 )             ;    "      "    "  0
```

<EXIT> factorial 1 ; exit 0! = 1 <EXIT> factorial 1 ; exit 1! = 1 <EXIT> factorial 2 ; exit 2! = 1 <EXIT> factorial 6 ; exit 3! = 6 <EXIT> factorial 24 ; exit 4! = 24 <EXIT> factorial 120 ; exit 5! = 120 120

```
-->(untrace factorial)                  ; ask LISP to shut up
```

(showstack)

When called after an error causing entry to the break level will display the last 20 evaluations including the one which caused the error. The top of the internal stack is copied

```
whenever  LISP is about to enter the break level (prompt  'er>').
```
~~~~~~~~~~~~~~~~~~~~~~~~~~

This means that if you execute some function and it aborts prematurely you can call showstack from the break level and see exactly what lead to the error. Whenever a new error occurs the old copy of the top 20 elements on the internal stack is lost and a new trace is copied for you to display via (showstack). This is unlike Franz which allows lots of break levels. For example consider this example session with PC-LISP which is similar to an example in LISPcraft.

```
-->(defun foobar(y)(prog(x)(setq x (cons (car 8) y]
```

foobar

```
-->(foobar '(a b c))
```

--- error evaluating built in function [car] ---

```
er>x
()
er>y
(a b c)
er>(showstack)
```

[] (car 8) [] (cons <**> y) [] (setq x <**>) [] (prog(x) <**>) [] (foobar '(a b c))

t

In this example I declared a function called 'foobar' which runs a prog and does a single assignment to x. When I execute it with parameter '(a b c). PC-LISP correctly tells me that there was an error evaluating the built in function 'car'. I can examine the values of x and y and see that x is still set to the empty list () that the prog call set it to. y is bound to the parameter passed to foobar as expected. Next I called (showstack) to see the trace of execution. I see that the top evaluation (car 8) is the culprit. The <**> symbols in the show stack are just a short hand way of saying look at the entry above to see what the <**> should be replaced with. This greatly reduces the amount of information that you have to look at when you read a stack dump. It also allows you to follow the stream of partial evaluations by looking at each <**> in turn. Note that infinite recursion leaves a telltale stream of <**>'s.

(sstatus *a1* *s1*)

Returns t but has the side effect of setting system option a1 to setting s1. The legal values of a1 are symbols or strings with print names in the following list: smart-backslash, ignoreeof, chainatom, and automatic-reset. Any S-expression is legal for s1 but it is only tested for nil or non nil. The effects are as follows.

```
(sstatus smart-backslash nil) will cause \n \t \r etc.  in a
```

string or delimited symbol to be interpreted as n,t,r etc. On the other hand (sstatus smart-backslash t) causes the \n \t \r etc. sequences to be interpreted as newline, tab, carriage return as described in the section on SYNTAX (this is the default).

```
(sstatus ignoreeof nil) will cause an exit to occur when the
```

EOF sequence is typed on the console from the top level. This is the default. On the other hand (sstatus ignoreeof t) will cause an EOF sequence typed on the console from the top level to be ignored. This is used to protect against accidental exit from the top level (exit must be done by evaluating (exit) explicitly).

```
(sstatus  chainatom nil) will cause an error to  occur  when
```

either (car) or (cdr) of an object that is not a list is evaluated. This is the default. (sstatus chainatom t) will cause nil to be returned by (car) and (cdr) if they are evaulated with a non list parameter.

```
(sstatus automatic-reset nil) will cause entry to the  break
```

level to occur after an error is detected (this is the default). But, (sstatus automatic-reset t) will cause the break level to be skipped and no bindings will be held. It is as if you typed the EOF sequence from the break level after every error. This is useful in Franz where break levels can go N deep, it has limited use in PC-LISP.

28. List Evaluation Control Functions

These functions are the control flow functions for LISP they affect which lists are evaluated and how. They operate on the basic LISP function types, descriptions of which follow.

(lambda l1 s1....sn)

This is not a function but it is a list construct which can act as a function in any context where a function is legal. A lambda expression is a function body. The S-expressions s1..sn are expressions that are evaluated in the order s1...sn. The result is the evaluation of sn. The atoms in the list l1 are called bound variables. They will be bound to values that occur on the right of the lambda expression before the S-expressions s1..sn are evaluated and unbound after the value of sn is returned.

(nlambda l1 s1....sn)

This is a function body construct similar to lambda but with a few major differences. The first is that the list l1 must only specify one formal parameter. This will be set to a list of the UNEVALUATED parameters that fall on the right of the nlambda expression when it is being evaluated. This function allows you to write functions with a variable number of parameters and to control the evaluation of these parameters. For example we can write a function called 'ADDEM that behaves the same way as '+ in nearly all contexts as follows:

```
-->(def ADDEM (nlambda(l)(eval(cons '+ l))))
```

or

```
-->(defun ADDEM fexpr(l)(eval(cons '+ l)))
```

Both of which create the same nlambda expression. This function will behave as follows when spotted on the left of a sequence of parameters 1 2 3 4. First it will not evaluate the sequence of parameters 1 2 3 4. Second it makes these into a list (1 2 3 4). It then binds 'l to this list and evaluates the expression (eval(cons('+ l))). This expression results in (eval (+ 1 2 3 4)). Which is just the desired result 10.

```
(label a1 (lambda|nlambda l1 s1..sn))      {not in Franz}
```

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ This acts just like a lambda expression except that the body is temporarily bound to the name a1 for evaluation of the body s1. This allows recursive calls to the same body. The binding of the body to the name a1 will be forgotten as soon as the expression s1 terminates the recursion. For example:

```
(label LastElement (lambda(List)
                   (cond ((null (cdr List))(car List))
                         (t (LastElement (cdr List))))))
```

```
(lexpr (a1) s1 s2 .... sN)
```

~~~~~~~~~~~~~~~~~~~~~~~~ This function body form is similar to the nlambda form except that all of its variable number of arguments are evaluated and the args are accessed in a different manner. The second element of the lexpr form must be a list of exactly one atom. The remaining elements of the lexpr form represent bodies that are evaluated one after the other. The result of evaluating a list whose first element is an lexpr is just the value that results from evaluating s1....sN in order in the context where a1 is bound to the number of actual parameters, and the (arg), (setarg) and (listify) functions behave as follows:

(arg [n1])

When in the context of an lexpr's evaluation, will return either the number of arguments provided to the nearest enclosing lexpr, or the nth argument indexed from 1 passed to the lexpr depending on whether or not n1 is provided as a parameter. An error occurs if n1 is less than 1 or greater than (arg).

(setarg n1 s1)

When in the context of an lexpr's evaluation, will return exactly s1. It has the side effect that future calls to (arg n1) will return the value s1 for the duration of the current enclosing lexpr evaluation. An error occurs if n1 is less than 1 or greater than (arg).

(listify n1)

When in the context of an lexpr's evaluation, will return a tail of the list of arguments that were passed to the nearest enclosing lexpr. The head of this tail is either (arg n1) if n1 is positive, or (arg (+ (arg) n1 1)) if n is negative. If the value of n1 does not correctly index a head within the actual argument list, nil is returned.

Here is a small lexpr example which just sets some global variables to allow us to see what went on inside. Again see LISPcraft for a much better description.

```
-->((lexpr(n)
        (setq a0 n a1 (arg 1) an (arg(arg)))
        (listify -3)
```

) 'A 'B 'C 'D 'E 'F 'G)

```
(E F G)
-->a0
```

7

```
-->a1
```

A

```
-->an
```

G

(apply s1 l1)

The function s1 is evaluated in the context resulting from binding its formal parameters to the values in l1. The result of this evaluation is returned. Example:

```
-->(apply '(lambda(x y z)(* (+ x y) z)) '(2 3 4))
```

20

(cond l1 l2 ... ln)

The lists l1 ... ln are checked on by one. They are of the form (s1 s2 .. sn). Cond evaluates the s1's one by one until it finds one that does not eval to nil. It then evaluates the s2..sn expressions one by one and returns the result of evaluating sn. If all of the s1's (called guards) evaluate to nil, it returns 'nil. For example:

```
-->(cond ((equal '(a b c) (cdr '(x a b c))) 'yes)
          (t 'opps))
```

yes

(eval s1)

Runs the LISP interpreter on the S-expression s1. It is like removing a quote from the expression s1. For example:

```
-->(eval '(+ 2 4))
```

6

```
(mapcar s1 l1 l2 l3 .... ln) and (mapc s1 l1 ... ln)
```

~~~~~~ These functions will map the function s1 onto the parameter list made by taking the car of each of l1...ln. It forms a list of the results of the repeated application of s1 to the next elements in the lists l1...ln. It stops when the list l1 runs out of elements. Note that each of l1...ln should have the same number of elements, although this condition is not checked for and nil will be substituted if a list runs out of elements before the others. Extra elements in any list are ignored. For example:

```
-->(mapcar '< '(10 20 30) '(11 19 30))
(t nil nil)
```
~~~~~~

Which returns the results of (< 10 11) (< 20 19) and (< 30 30) as the list (t nil nil).

The function mapc operates in exactly the same way as mapcar except that the result is just l1. No result list is returned. Mapc is meant to be used to save a little memory when the side effect is of interest, not the list of returned values from each individual mapping.

```
(maplist s1 l1 l2 ... ln) and (map s1 l1 l2 ... ln)
```

These functions are similar to mapc and mapcar except that rather than taking successive cars down the lists l1...ln to make argument lists, they take successive cdrs down the lists l1...ln to make the arguement lists. Map does the same thing as maplist but it does not construct a list of the results of each mapping, rather it just returns l1. Like mapc, this is usefull when the side effect is of greater interest than the result. For example:

```
-->(maplist 'cons '(a b) '(x y))
(((a b) x y) ((b)(y))
-->(maplist 'car '(a b c))
(a b c)
-->(defun silly(a b)(list a b))
-->(trace silly)
(silly)
-->(mapc silly '(a b) '(c d))
<enter> silly((a b) (c d))
```

<EXIT> silly ((a b) (c d))

```
<enter> silly((b)(d))
```

<EXIT> silly((b)(d))

```
(a b)
```

These examples operate as follows. The first example simply cons'es the list (a b) to the list (x y) and then cons'es the list (b) to the list (y). It then makes a list of these two results and returns it (((a b) x y) and ((b) y). The second applies car to successive cdr's of the list (a b c) ie it evaluates (car (a b c)) then (car (b c)) then (car (c)) and makes a list of the results (a b c) which it returns. The last example demonstrates that mapc does the same thing as maplist except that there is no list of results returned. Rather the first argument list is returned. To demonstrate this the function 'silly which makes a list of its two arguments was traced as it was mapped accross the argument lists '(a b) and '(c d). This results in silly being called with two lists as parameters the first time, and the second time with the cdr's of the above two lists.

Note the functions mapcon and mapcan are NOT built into PC- LISP but are available in PC-LISP.L as macros. The extra functions mapcon and mapcan apply (nconc). Use them carefully. (See the notes on (nconc) in the DANGEROUS FUNCTIONS section).

(defun a1 macro l1 s1 s2 ... sn)

Macro is a special body, similar to nlambda except that it may causes code replacement when it is evaluated. When a macro is encountered the list (name arg1 arg2...) is bound to the macro parameter l1. Name is the name of the macro and arg1..argn are the arguments that were provided to it. Then the bodies of the macro s1..sn are evaluated and the expression returned by the last body is returned. Then depending on the value of displace-macros and the type of the returned S-expression, the returned S- expression may destructively replace the peice of code that called it. If the value of displace-macros is nil (its default value) or the type of the returned S-expression is not one that can be replaced, no destructive substitution will occur. Next regardless of whether the S-expression was substituted or not, the S-expression is evaluated and the value returned. This all sounds pretty compex, but in fact it is quite simple, here is an example:

```
-->(defun first-elemet macro(l)(cons 'car (cdr l)))
```

```
first-element
          -->(setq x '(first-element '(a b c)))
          (first-element '(a b c))
          -->(eval x)
a
          -->x
          (first-element '(a b c))
          -->(setq displace-macros t)
t
          -->(eval x)
a
          -->x
          (car '(a b c))
          -->(eval x)
a
```

In the example above I have first declared a macro called 'first-element' which when run given a list parameter should return the first element in the list. I could have done this using a lambda expression but this would require parameter binding etc every time I execute 'first-element'. Rather, what I have chosen to do is to cause (first-element x) to be replaced by the code (car x) everywhere it is encountered. Then future execution of (first-element x) is just as costly as an execution of (car x). Let's examine what I did above. First I declared a macro which will take the parameter (first-element -stuff-) and construct the code (car -stuff-). I then set x to be an expression which when evaluated should give 'a. I then verify this by evaluating x, sure enough it is 'a. I then look at the code for x which has not changed. Now, I set the global variable displace-macros to be non nil. What I should now expect is that (eval x) will give the same answer, but with the side effect of doing the code substitution so that future passes of the

expression bound to x will run much faster. This is the whole reason for macros, they are not much use if they are expanded every time, it is more work than a simple user defined lambda expression call. Anyway after running x and looking at its definition we can see that the code has indeed been substituted. It is worth noting that unless you set displace-macros to be non nil all your macros will be expanded every time they are encountered. This is probably not what you want. You should set displace-macros to be t to cause macros to behave properly. The only reason I did not set displace-macros to be t by default is that Franz does not.

Note, macros may return any type expression however some expressions may not result in code substitution because of internal problems with doing the substitution. In particular a macro that directly returns an atom, hunk or string will never result in code replacement, while a macro that returns a list, fixnum, flonum or port can result in code replacement. Since code replacement is a physical copying of one cell over another heap space owning functions cannot be physically substituted because their cells are unique. You should note however that these limitations do not occur much in practice since usually a macro will return a number or a list. For exampe a quoted atom is ok because it is really the list (quote x). In any case PC-LISP macros will always return the correct values regardless of these substitution limitations.

Macro bodies can function in all contexts that an nlambda body can function, however expansion, if it is to occur will only happen when a macro is referred to by its atom name which was defined by a defun, def or putd call. Using macro expressions disembodied from a name does not however seem terribly useful.

(macroexpand s1)

This function lets you see what the macro expansion of s1 looks like prior to evaluation and substitution. This function is useful for debugging macro definitions and for controlling macro evaluation when writing code that

generates new code.

```
-->(macroexpand '(a b (first-element '(a b c)) x y z))
(a b (car '(a b c)) x y z)
```

Macroexpand will expand all macros in s1 but will not expand lists that start with quote. The workings of macroexpand are probably a little different than Franz although the results should be pretty much the same. Note in particular that macroexpand creates a new structure, it does not expand into the existing structure as (eval) does during real macro expansion.

(prog l1 s1.....sn)

Prog is a way of escaping the pure LISP applicative programming environment. It allows you to evaluate a sequence of S-expressions one after the other in true imperative style. It allows you to use the functions (go..) and (return ..) to perform the goto and return functions that imperative languages permit. Prog operates as follows: The list l1 which is a list of atom names is scanned and each atom is bound to nil at this scope level. Next the S-expressions s1..sn are scanned once. If any of s1..sn are atoms they are bound to the S-expression that follows them. Next we start evaluating lists s1...sn ignoring the atoms which are assumed to be labels. If after evaluation an S- expression is of the form ($[|return|]$ Z) we unbind all the atoms and labels and return the S-expression Z. If after evaluation a list is of the form ($[|go|]$ Z) we alter our evaluation to start next at Z. The functions (go) and (return) will return the above mentioned special forms. If at any time we reach sn, and it is not a go or a return, we simply unbind all of l1 and the labels in s1...sn and return the result of evaluating sn. Note that prog labels must be alpha or literal alpha atoms. You are advised to keep the calls to go and return within the lexical scope of the prog body and to ensure that the special form returned is not absorbed by some higher level function.

```
-->(prog (List SumOfAtoms)
         (setq List (hashtabstat))
         (setq SumOfAtoms 0)
```

LOOP (cond ((null List) (return SumOfAtoms)))

```
         (setq SumOfAtoms (+ (car List) SumOfAtoms))
         (setq List (cdr List))
         (go   LOOP)
```

) 306

This peice of code operates as follows. First it creates two local variables. Next it binds the variable List to the list of hash bucket totals from the heap hash table. It then sets a sum counter to 0. Next it checks the List variable to see if it is nil. If so it returns the Sum Of all the Atoms. Otherwise it adds the first fixnum in the list List to the running SumOfAtoms, winds in the list List by one, and jumps to LOOP. Note also that we can accomplish the same thing as the above prog with the much simpler example which follows:

```
-->(eval (cons '+ (hashtabstat)))
```

306

If execution reaches the end of the prog without encountering a (return), the last evaluated expression is returned.

(caseq exp *l1* *l2* ... *lN*)

Where l1..lN are of the form (key s1...sN) or ((key1 key2 ..keyN) s1...sN). Will select one of a number of cases l1..lN. The case will be selected if its key or one of the keys in a key list are eq to 'exp'. The key 't will match anything. When a case is selected, the expressions s1...sN are evaluated left to right and the result of the last evaluation is returned. For example:

```
-->(caseq 'apple
        (orange "orange")
        (grape  "green")
        ((stawberry cherry apple) "red"))
```

```
"red"
        -->(caseq '|\n|
                ((a b c d e f g h i j k l m n o p q r s t u v w x y z)
'lowercase)
                ((A B C D E F G H I J K L M N O P Q R S T U V W X Y Z)
'uppercase)
                (t 'other))
other
        -->(caseq (length '(a b c d))
                  ( 0 nil)
                  ( 1 nil)
                  ( 2 (setq x 2) (patom "length is 2\n") t)
                  ( 3 (patom "length is 3\n") t)
                  ( (4 5 6 7 8 9 10)
                      (patom "length is in 4..10\n") t))
length is in 4..10 t
        -->
```

Given the choice between caseq and cond, pick caseq because it is much faster than cond. This is because most of the work of testing conditions is done in the interpreter.

(funcall s1 .... sN)

Will apply the function s1 to arguments s2..sN and return the result. This is equivalent to (apply s1 (list s2...sN)). The function s1 must be present but the arguments s2 .. sN are optional and depend on the number of arguments/discipline of s1.

```
        -->(funcall 'car '(a b c))
a
        -->(funcall 'cons 'a '(b c))
        (a b c)
```

29. Error Trapping And Generation

PC-LISP V2.16 provides the user with the ability to trap any error with the exception of stack overflows (for technical reasons). This is accomplished by the (errset) and (err) functions. Similar control flow violations are provided for general use via the (catch) and (throw) functions.

(errset *s1* [*s2*])

Will evaluate its argument s2 THEN s1, if s1 results in any kind of error (with the exception of a stack ovflow), then errset will return the value nil. If the value of s2 is non nil then the normal error message is printed at this time on the console. If no error occurs in evaluating s1 then the result of evaluating s1 is made into a list and returned (to distinguish nil from (nil)). If the error was generated by a call to (err s1) then the value s1 is returned rather than nil. Any bindings that were made between the time (errset) was called and the point of the error will be undone. However, global bindings are not undone. The number of (errset)s that can be nested is determined by the amount of memory you have, there is no arbitrary fixed limit. For example:

```
        -->(errset (car (cdr (car (car 8]
```

```
--- error evaluating built in function [car] --- nil
          -->(errset (car 8) nil)        ; car err msg not printed.
nil
          -->(errset (atom 8))           ; no error
          (8)
          -->(errset (prog () l (go l)) nil)  ; trap CTRL-BREAK.
nil
```

(err s1)

Will 'throw' the value of s1 to the nearest enclosing errset which will then return with the value s1. If no errset encloses the evaluation of err then the break level is entered and the message "--- user err ---" is displayed. For example:

```
          -->(errset (+ 1 2 3 (err 'x)) nil)
x
          -->(errset (+ 1 2 3 (err 'x)))
--- user err --- x
          -->(err 'x)
--- user err ---
          er>                  ; now do an end of file CONTROL-Z
          -->(errset (errset (+ 1 2 3 (err 'x)) nil))
          (x)
          -->                  ; the 2nd errset trapped, 1st did not.
```

30. Non Standard Control Flow Functions

PC-LISP V2.16 provides a means of skipping the returns to outer evaluations and 'throwing' a value up to an outer routine to be returned by that routine. These two routines are called (throw) and (catch) respectively. They operate in a manner similar to (err) and (errset) but allow selective catching by means of a tag assocaited with a thrown value.

(catch *s1* [*s2*])

Will evaluate s2 first, then s1. If during the evaluation of s1 a call is made to (throw s3) the catch will return immediately with the value s3. If s2 is provided it is interpreted as a tag, or a list of tags (where a tag is a symbol). The catch is then selective in the throws that it will catch. It will not catch the thrown expression unless the tag argument provided to the throw matches either the symbol s2, or one of the symbols in the list s2. The symbol nil matches all tags. If the tag of the thrown expression does not match the symbol s2, or is not present in the list of symbols s2, the expression and tag will be throw up to the next closest enclosing catch. If a throw arrives at the top level having not been caught, the error handler will catch it and display the message "--- no catch for this tag [xyz] ---". Where xyz was the tag assocaited with the thrown expression. This error like all other errors (with the exception of a stack overflow) can be trapped by (errset). As with errset and err, all bindings other than global bindings that were made between the time the catch was called and the throw was called will be undone. As with errset, there is no fixed limit to the depth of nesting.

(throw s1 [s2])

Will never return, it throws the expression s1 with tag s2 or nil if s2 is not provided up to the nearest enclosing (catch). The catch will either catch the expression or throw it up to the next enclosing (catch) depending on whether the tag s2 matches the catch's tag or list of tags. A nil tag (the default) will match any other tag. If the thrown expression is not caught by any (catch) the error handler will catch it and generate the error described for

(catch) above. For example:

```
            -->(catch (patom "hi" (throw 'x)))
x
            -->(catch (patom "hi" (throw 'x)) 'MyTag1)
x
            -->(catch (patom "hi" (throw 'x)) 'MyTag1) 'MyTag1)
x
            -->(catch (patom "hi" (throw 'x)) 'MyTag1) '(a b MyTag1))
x
            -->(catch (catch "hi" (patom (throw 'x 't1)) 't2) 't1)
x
            -->(catch (patom "hi" (throw 'x))) 'MyTag1)
```

--- no catch for this tag [MyTag] ---

31. Hunks

A hunk is just an array of 1 to 126 elements. The elements may be any other type including hunks. With hunks it is possible to create self referencial structures (see DANGEROUS FUNCTIONS). A Hunks element storage space comes from the heap. Hunks like strings and alpha print names are subject to compaction relocation and reclaimation.

(hunk s1 s2 .... sN)

Returns a newly created hunk of size N whose elements are s1, s2 ... sN in that order. N must be in the range 1 to 126 inclusive. Note that a hunk is printed like a list but is delimited by { } not (). Ie {s1 s2 ...sN}.

(cxr n1 H)

Returns the n1'th element of hunk H indexed from 0. Hence n1 must be in the range 0 .. (hunksize H)-1.

(hunkp s1)

Returns true if s1 is of type hunk, otherwise it returns nil. Note this function has also been mentioned with the other predicates.

(hunksize H)

Returns a fixnum whose value is the size of the hunk. This value is one larger than the largest index allowed into the hunk by both cxr and rplacx. The size of a hunk is fixed at the time of its creation and can never change throughout it's life.

```
            (makhunk n1) or (makhunk (s1 s2 ...sN))
```

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ The first form returns a nil filled hunk of n1 elements. Needless to say, n1 must be between 1 and 126 inclusive. The second form is just identical to (hunk s1.....sN).

(rplacx n1 H s1)

Returns the hunk H, however as a side effect element n1 of H has been made (eq) to s1. In other words H[n1] = s1. Note that this function like rplaca and rplacd allows you to create self referencial structures.

32. Arrays

(array *a1* *a2* n1 ... nN)

Allocates and returns a nil filled array whose name is a1 and whose dimensions are n1 x n2 x..nN. Parameter a2 is ignored and is there for compatability with MacLisp arrays. There must be at least one dimension. The total size of the array is only limited by the amount of memory available. After a call to (array a1..) the symbol a1 will behave like a function that accesses the array using it's args as dimensions and optional element to
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

store in the array. (See next function).

(A [exp] n2... nN)

If A is the name of an array created by (array a1 t n1...nN) then this function will return the element corresponding to indecies n1...nN. If exp is provided then the element corresponding to indecies n1...nN will be set eq to exp. It is an error not to provide legal indecies to an array.

(arraydims A)

Returns a list whose car is t, and whose cdr is the list n1...nN that was provided to (array) when the array A was created. In other words returns a list whose car is the type of the elements of the array, and whose cdr is a list of the dimensions of the array.

(getlength A)

Returns a fixnum whose value is the size of the array. This is computed as n1xn2xn3...xnN where n1...nN are the dimensions provided in the (array) call that created the array A.

(getdata A)

Returns a hunk that is the root of the array tree used to store the raw elements of A. Its structure is given in the section on data types in this manual.

(listarray A [n1])

Returns a list of the elements in A. If n1 is provided then the first n1 elements of A are placed into the returned list.

(fillarray A l1)

Fills the array A with successive elements of l1. If l1 has less elements than A, the last element in l1 is used to fill the remainder of the elements in A.

Note that like hunks arrays allow us to create self referential structures so watch out. Here is a short example of how the array access functions work together. Note that (store) is a macro included in PC-LISP.L. You should be able to type these same statements and get the same results.

```
          -->(array A1 t 20 100)          ; A1 is a 20 x 100 array
array[2000]           ; ie it has 2000 elements
          -->(A1 0 0)                     ; get A1[0,0], it is nil
nil             ; to start with.
          -->(A1 "hello" 0 0)             ; A1[0,0] = "hello"
"hello"
          -->(store (A1 19 99) "there")   ; macro A1[19,99]="there"
"there"
          -->(A1 19 99)
"there"
          -->(arraydims 'A1)              ; get the dimensions of A1
          (t 20 100)                      ; 20 x 100, any type elems
          -->(getlength 'A1)
2000
          -->(listarray 'A1 5)            ; make list of first 5
          ("hello" nil nil nil nil)
          -->(getd 'A1)
```

array[2000]

```
-->(type (getdata 'A1))          ; getdata give hunk tree.
```

hunk

Note that if you try to allocate an array that is too big for the available memory you will get either an out of heap, or out of cons cell error message followed by entry to the break level. Try making your LISP_HEAP and/or your LISP_ALPH environment variables smaller. (See also memory exhaustion).

33. Dangerous Functions

The following two functions have potentially disasterous results if used by unwary or inexperienced LISP programmers. The third function is provided to make their use less dangerous.

(rplaca l1 s1)

The cons cell l1 is physically altered so that its car is (eq) to s1. That is the car pointer of l1 is set to point to s1. The list l1 is returned. (l1 must not be nil).

(rplacd l1 s1)

The cons cell l1 is physically altered so that its cdr is (eq) to s1. That is the cdr pointer of l1 is set to point to s1. The list l1 is returned. (l1 must not be nil).

(copy s1)

Returns a structure (equal) to s1 but made with new cons cells. Note that only cons cells are copied, strings, atoms, hunks etc are not copied.

Warning #1 - altering a cons cell allows you to create structures that point (refer) to themselves. While this does not cause a problem for the LISP interpreter or garbage collector it does mean that many built in functions will either loop around the structure infinitely or recurse until a stack overflows.

```
-->(setq x '(a b c d))
(a b c d)
-->(rplaca x x)
((((((((((((((((((((((((((((((((((((((((.............
```

-- stack overflow --

```
er>
```

Warning #2 - altering a cons cell can cause a million little side effects that you did not count on. Consider carefully the following example.

```
-->(defun FooBar(x) (cons x '(b c)))
```

FooBar

```
-->(setq z (FooBar 'a))
(a b c)
-->(rplaca (cdr z) 'GOTCHA!)
(GOTCAH! c)
-->(FooBar 'a)
(a GOTCHA! c)
```

What happened? The rplaca has modified the list that is a constant in FooBar. Lists are not copied unless necessary and building the list (a b c) did not require a copy of the constant list (b c) to be made. Ie (cdr z) is eq to (b c) in FooBar.

(sort l1 s1)

Destructively sorts the list l1 and returns it. The function s1 is used to compare elements in the list. If s1 is nil the function alphalessp is used. If s1 is non nil, it must be a lambda expression taking two parameters and

return t or nil. It may also be the name of a built in function such as '< or '>. etc. For example:

```
-->(sort '(john frank adam) nil)
(adam frank john)

-->(sort '(10 9 8 1 2 3) '<)
(1 2 3 8 9 10)

-->(sort '(10 9 8 1 2 3) '(lambda(x y)(not (< x y))))
(10 9 8 3 1 2) ; reverse of last example would be faster.
```

(sortcar l1 s1)

Destructively sorts the list l1 and returns it. The car of each element in l1 is used as the key rather then the element itself as in (sort) above. The function s1 is used to compare elements. s1 is as in (sort). For example.

```
-->(sortcar '( (john smith) (frank jones) (adam west)) nil)
((adam west)(frank jones)(john smith))
```

Note that these functions are destructive, they alter the actual list parameters passed to them. Either make sure you know what you are doing, or use (copy l1) before passing the list as a parameter to (sort) or (sortcar). Both of these functions use the quicksort algorithm. Ie, they run in O(n*lg(n)) time. Note that there is a limit to the length of list that you can sort imposed by the size of the system stack. Sorting with s1=nil, is much much faster than providing your own compare routine. If you want the reverse ordering, sort using s1 = nil, then call (reverse) don't do (sort l1 '(lambda(k1 k2)(not(alphalessp(k1 k2)))), it will be much slower. Sorting with s1=nil also does not cause any garbage collection whereas sorting with s1 not = nil may be interrupted by garbage collection because of the overhead of building a parameter list and calling the function.

(nconc l1 l2 ... ln)

Similar to append except that the lists are joined together destructively. The last cons cell in l1 is changed so that its cdr points to l2, the last cons cell in l2 is changed to point to l3 etc. Note that any nil parameters are ignored. Nconc returns the constructed list or nil if all parameters are nil.

34. Msdos Bios Calls For Graphics Output

These functions allow you to perform BIOS level graphics/character oriented I/O. They all result in an INT 10H. This means that the graphics should be portable to most MSDOS machines and should run under any windowing environment like Topview or MSwindows. This is why they are so slow. Note that they all return 't. They do not check to see if the INT call was successful or if you have a graphics capability. You can crash your system if you abuse these functions.

```
(#scrline# n1 n2 n3 n4 n5)
```

~~~~~~~~~~~~~~~~~~~~~~ Draws a line on the screen connecting (n1,n2) with the point (n3,n4) using attribute n5. BIOS level I/O means this is slow but does work on every MS-DOS machine I know of!

```
(#scrmde# n1)        {ah=0, al=n1, INT 10H}
```

~~~~~~~~~~ Sets the video mode to n1. Modes are positive numbers 0..... Where (8 and 9) are high resolution for the Tandy2000 and I suppose are high resolution modes on other machines that support the (640 x 400) or greater graphics resolutions. These are all listed in your hardware reference manual but basically they are: 0 = 40x25B&W, 1=40x25COL, 2=80x25B&W 3=80x25COL, 4 =320x200COL, 5=320x200B&W, 6=640x200B&W, 7=reserved, 8=640x400COL, 9=640x400B&W etc...? This is as of DOS 2.10. Also note that the AT EGA Graphics Modes should also work with no problem. The value of n1 is not checked. This allows for the unpredictably high modes required by some machines.

```
(#scrsap# n1)       {ah=5, al=n1, INT 10H}
```

~~~~~~~~~~~ Sets the active video page to n1. n1 should be between 0 and 8. This is valid for text modes only. Versions of MSDOS other than 2.10 may not support this call.

```
(#scrspt# bh bl al)       {ah=11,bh=bh,bl=bl,al=al,INT 10H}
```

~~~~~~~~~~~~~~~~ Sets the color palette according to the value in bh. For most BIOS compatable machines these are: If bh=0 it sets background color bl. If bh=1 it sets the default palette to the number 0 or 1 in BL. If bh=2 it sets a single palette entry where bl is the palette entry number and al is the color value. See your BIOS reference for the color values and additional info.

```
(#scrscp# n1 n2 n3)       {ah=2,bh=n1,dh=n2,dl=n3,INT 10H}
```

~~~~~~~~~~~~~~~~ Sets the cursor position to be in page n1 at row n2 and in column n3. Where 0 is the top row and 0 is leftmost col.

```
(#scrsct# n1 n2)          {ah=1,ch=n1,cl=n2,INT 10H}
```

~~~~~~~~~~~~~ Sets the cursor type to agree with the following: n1 bit 5 (0 = blink 1 = steady), bit 6 (0 = visible, 1 = invisible), bits 4-0 = start line for cursor within character cell. n2 bits 4-0 = end line for cursor within character cell.

```
(#scrwdot# n1 n2 n3)      {ah=12,cx=n1,dx=n2,al=n3,INT 10H}
```

~~~~~~~~~~~~~~~~~~ Write a dot (pixel). The pixel at row n1 and column n2 has its color value XORed with the color attribute n3. Since the color attributes vary from machine to machine you will have to look up the correct values for this parameter in your BIOS guide.

35. Memory Exhaustion

The memory is all used up when you get a message such as "--- out of cons cells ---". Usually when this happens it is because you are tying up memory somewhere but do not realize it. The most common way to tie up memory is to execute an infinite recursion such as (defun looper(n)(looper (+ n 1))). The stack will of course overflow and YOUR BINDINGS WILL BE HELD FOR YOU!! This means that ALL bindings are held. If you execute the above

```
program several times from the break level, 'er>', you will
```

eventually run out of CONS cells. They are all in use to hold the values n, n+1, n+2,..... to the point of the first stack overflow. Then n, n+1,.... to the point of the second overflow and so on and so on. Eventually there is no more space left to evaluate the function (looper). The solution is simple: If you run an infinite recursion by mistake and are placed in the break level, use the showstack to figure out where you are. Then use the break level to examine variables etc. But before retrying anything return to the top level. This will cause the held bindings to be dropped and the cells will become reclaimable ie (garbage and thus free). Consider the following session with PC-LISP V2.16:

```
-->(defun looper(n)(looper (+ n 1)))     ; infinite function
looper
-->(looper 0)                            ; run it from 0
-- Stack Overflow --            ; all n's saved!
er>n                                     ; last value of n
588
er>(looper 0)                            ; another run will
-- Stack Overflow --            ; save more n's
er>(looper 0)
```
~~~~~~~~~~~~~~~~~~

-- Stack Overflow --

```
er>(looper 0)                                   ; another run won't
```

--- out of cons cells ---

```
er>
```

Note that the last (looper 0) call we made from the break level was unable to complete because we ran out of memory. When any of the three types of memory is exhausted a message is printed and the break level is entered. In most cases it is possible to continue by typing CONTROL-Z and ENTER or CONTROL-D (if you are using UNIX) to return to the top level.

If you find that you are running out of heap space it may be because you are keeping too many unused strings,symbols or hunks. Or, you are trying to allocate an array that is too big for the amount or configuration of your H and A LISP_MEM settings. In the first case the solution is not to do things like (setq x (oblist)). In the second case the solution is to adjust the H option up and the A option down until the array can be allocated. There is of course a practical limit to the size of array that can be allocated on a 640K IBM-PC. A UNIX machine or MS-DOS machine without the 640K limit will be restricted by the hard limit of 75 allocatable blocks.

36. Technical Information

The interpreter is written 99% in C. The other 1 percent is assember needed to trap things like stack overflows and handle BIOS level graphics on an MS-DOS machines. The UNIX version requires no extra assembly language. In total the program is nearly 9000 lines of C and is easily ported to most UNIX machines but requires a little assembler for most MS-DOS machines.

Memory is organized as follows. Alpha cells have fields for a shallow stack of bindings, a pointer to heap space for the print names, a pointer to any built in or user defined functions, and a pointer to any property lists. Alpha cells are the largest of all the cells and have their own fixed storage area. Heap space which is just the space used for the print names of the alpha cells and strings, and the element array for hunks may be variable sized blocks of up to 254 bytes long. This is why a hunk can have only 126 elements in PC-LISP. The rest of the cells used by PC-LISP are all considered as one. This consists of the flonum, fixnum, list, string, hunk and port cells. They have their own contiguous slice of memory. This means that three different contiguous types of memory are required. It is managed in the following way. At start up time the percentages of memory are read from the default settings or the environment variables LISP_MEM. Next memory is allocated in 16K chunks up to either the limit given by the B setting in the LISP_MEM variable, or until either no memory is left or the hard limit of 75 blocks is reached. These blocks are then kept track of in a large vector of pointers. Next groups of these blocks are primed for use by alpha,cell, or heap managers according to the A and H settings in the LISP_MEM environment variable or the default of 1 each. These managers handle the distribution and reclamation of memory in their own block. The heap manager will perform compaction and relocation to get free space. The alpha and cell managers will perform mark and gather garbage collection to get space. The heap manager may request mark and gather collection if there is a real shortage of heap space prior to performing compaction.

Stack overflow detection is done by intercepting the call to the MSC __chkstk() routine. And performing its usual function of local storage allocation but when an overflow occurs temporarily resets the stack, and them making a call to my own C stack overflow routine. This then longjmps out of the error condition. The UNIX version does not require this checking because an internal stack (not the C stack) will always overflow first. The opposite is always true of the MS-DOS version.

Control-BREAK detection is done via periodic testing of the status in the evaluator main loop, and the read main loop. When a break is detected control is transferred to the break handler which prints a message and longjmps back to the mainline code. The MS-DOS version must poll the break status because DOS is not reentrant. The UNIX version also polls the break status but only because it is forced to by the logic of the MS-DOS version. CONTROL-C checking is done in the same way except that a CONTROL- C will only be spotted on I/O so a looping non printing function can only be stopped with CONTROL-BREAK. Note that CONTROL-BREAK is INT 1BH and CONTROL-C is INT 23H on an MS-DOS machine.

37. Known Bugs Or Lacking Features Of V2.16

-It is possible (but pretty unlikely) to run out of stack space while garbage collecting. When this happens the garbage collection is retried once but the error is unrecoverable. You should treat this as a stack overflow CAUSED BY YOUR PROGRAM. PC-LISP V2.16 uses a link inversion marking phase and thus uses a small bounded amount of stack space for garbage collection. Note that if the stack overflows on the second garbage collection retry PC-LISP gives up. Memory will be corrupt and you should quit because of the possibility of clobbering programs in RAM other than PC-LISP, ie DOS. If you CTRL-Z out of the error, no corruption will occur, but (exit) just could corrupt RAM if you were very unlucky.

-Two special atoms with rather obscure names should never be directly returned or manipulated in a prog. These are $[|return|]$ and $[|go|]$. If you attempt to say print these from within a prog, the print function will return them and this will confuse the heck out of prog which uses them for internal purposes. Because of this the (oblist) call does not return them. Thus the only way they can get into your code is for you to enter them directly. Since this is unlikely and I have warned you the problem should not occur.

-You are not prevented from altering the binding of t. This means that if you use t as a parameter or set/setq it to something other than t you may cause some strange behaviour, especially if you bind t to nil by accident. To limit this problem (defun) and (def) will check their parameter list for t or nil parameters before putd'ing the expression. (putd) however does not check!

-Macros are slightly restricted in that only lists, fixnums , flonums or ports can be substituted. This is a small difference from Franz but one that would require significant performance penalties to implement. Since not substituting these types is less expensive than implementing substitution would be, I will not implement this feature of Franz in a PC environment.

-Integer overflow/underflow is not trapped. The answers will silently change sign leaving you to figure out why your program does not work properly. These could be trapped but I have not figured out the best way to do it yet.

-A symbol with bindings should not be given a function definition and vise versa. This is because the binding of an atom is deemed to be its function body if it has no real binding. This is different from Franz and was done to simplify the evaluator. This is not really a problem because programmers are used to keeping function and variable names different.

-The interpreter is slow. I am planning on introducing a compiler which should speed things up significantly. The program is slower than some of the commercially available interpreters for 3 reasons. Mostly because it is a large model, many commercial interpreters are small models. This makes it nearly 3 times slower, but gives it more usable memory. PC-LISP uses 32bit integers which slow down many benchmarks. However, 32bit integers are what Franz provides and compatability is more important to me. Thirdly PC-LISP uses very little assembler as it is almost entirely C. A reasonable speed up could be achieved by rewriting one or two key procedures in assembler.

-Car and cdr will not access the first and second element of a hunk as they do in Franz.

-You cannot create custom array accessing schemes. I am not planning on introducing these features as they are probably used pretty infrequently and would make the already slow array accessing even slower.

-You cannot set the syntax of a character to anything other than a read or splicing read macro. I may introduce this stuff later on.

-Showstack does not print lists in compressed form horizontally. The vertical compression <**> is however done.

-Circular structures may cause problems for certain built in functions in particular you may not be able to abort the evaluation either. They do not however cause any problem for garbage collection and can be manipulated if you are careful.

-Depending on how much memory is free when PC-LISP is loaded it is possible that the (load) and (read) will become slowed down due to lack of buffer space for I/O. This happens very infrequently but if you notice the slowdown you can fix it by setting the LISP-MEM blocks value so that not all the free blocks are allocated. See the (exec) command for instructions on how to do this.

38. Re Bugs Or Desired Enhancments

Please let me know if you find a bug or if you have any suggestions. I am always interested to hear other peoples ideas and/or criticism.

Regards,

Peter Ashwood-Smith.

39. The Byte-Code Compiler

PC-LISP includes a byte-code compiler that translates LISP source forms into a compact stack-machine representation. The compiled form is executed by a dedicated byte-code interpreter that is roughly twice as fast as walking the original S-expression with **eval**. The compiler is exposed as ordinary LISP primitives, so it can be used interactively from the prompt or programmatically from within a load file.

39.1. The Pipeline

A compiled function passes through three stages, each available as a separate LISP primitive.

(compile expr)

Translates **expr** into an assembly-like S-expression of the form **($$clisp$$ nil <literals> <instructions>)** where **<literals>** is a symbol-table mapping each unique constant referenced by the code to a small index, and **<instructions>** is a list of mnemonic instructions and labels. The **expr** is normally a **(lambda**...) or **(nlambda**...) or **(lexpr**...) form, but any expression is acceptable. Side-effect-free built-in functions applied to constant arguments are folded at this stage; for example **(plus 1 2)** is replaced by the literal 3 before any byte code is emitted.

(ph-optimize clisp)

Runs a peephole pass over the assembly-language form returned by **compile**. It recognises a small number of redundant patterns (for example a push immediately followed by a pop, or a jump to the next instruction) and removes them. The function returns a list whose first element is the optimised **clisp** expression and whose remaining elements give the count of each kind of reduction performed. Use of **ph-optimize** is optional but recommended; the cost is negligible and the output is slightly smaller and faster.

(assemble clisp)

Takes the assembly-language form (either directly from **compile** or after **ph-optimize**) and converts it into a real byte-coded **clispcell** object. The output is a single S-expression of type *clisp* whose printed representation is **%L(addr),C(addr)%**. The cell holds two malloc-allocated arrays: a literal table referenced by the byte code, and the byte code itself. A **clispcell** may be bound to an atom with **putd**, passed as a function value to **apply**, or written to a file with **b-write**.

(disassemble clisp [port])

Prints a human-readable dump of a compiled **clispcell** to the given port (or to standard output if no port is given). Each instruction is shown with its address, mnemonic, raw byte values, and a comment indicating jump targets and the contents of any literal it references. The literal table is dumped after the code. If a literal is itself a compiled cell the disassembler descends into it recursively.

39.2. The Instruction Set

The byte-code interpreter understands roughly forty-six op codes covering five classes of operation: stack manipulation predicate tests hunkp ", " stringp ", " listp ", " numbp), list and number primitives dec ", " ddec ", " copyfix), control flow callnf ", " rcall ", " return), and the LISP-specific scope operations zpush ", " zpop ", " putclisp). The scope operations push and pop on each atom's shallow value-stack to implement dynamic binding inside a compiled function; **zpush** and **zpop** record enough information on the eval mark stack that a **throw** out of a compiled function correctly unwinds any bindings that the function had pushed. Each opcode is one byte and operands, when present, are 16 bits. The instruction set is documented in detail at the head of the **bucompil.c** source file.

39.3. A Worked Example

The following session shows the full compile pipeline applied to a simple recursive function. Comments in italics describe each step.

```
--> (defun fact (n)                          ; define an interpreted version
            (cond ((zerop n) 1)
                  (t (times n (fact (sub1 n))))))
fact

--> (compile (getd 'fact))                   ; -> assembly-language clisp
($$clisp$$ nil ((fact . 4) (1 . 3) (n . 2)) ...)

--> (assemble (compile (getd 'fact)))        ; -> byte-coded clispcell
%L(0xa31f10),C(0xa31fa0)%

--> (putd 'fact (assemble                    ; install the compiled body
                      (car (ph-optimize
                             (compile (getd 'fact))))))
fact

--> (fact 10)                                ; runs through the byte-code VM
3628800
```

39.4. Saving Compiled Code to a File

Compiled **clispcell** objects can be written to disk with **b-write** and read back with **b-read**. The **load** primitive auto-detects the binary format on a per-form basis: it peeks at the first byte of each top-level form and uses **b-read** when it sees a byte greater than 127, otherwise it falls through to the textual reader. A single file may therefore mix textual source and binary compiled forms, and either kind of file can be loaded with the ordinary **(load fname)** call. No special primitive or file extension is required to load a compiled file: whatever you would normally type to load the source you type to load the compiled version.

39.5. The compile-file Function

The convenience function **compile-file** (defined in the auto-loaded **pc-lisp.l**) performs the compile-and-write step in one call. Its signature is:

(compile-file in out)

It opens the source file *in* for reading, opens the output file *out* for writing, and walks the source one top-level form at a time. For each form:

- If the form is a **defun** or **defmacro** the function is first installed in the running interpreter (so that later forms in the same file can refer to it) and then run through the full pipeline **compile**, **ph-optimize**, **assemble**. The resulting compiled **clispcell** is wrapped in a **(putd 'name '<clisp>)** expression and written to *out* in binary using **b-write**.
- Any other form (including **setq**, **load**, top-level expressions, etc.) is written through to *out* unchanged in binary. Such forms will execute at load time exactly as they would have when the original source was loaded, so files that mix function definitions with side-effecting initialisation work correctly.

The function returns the name of the output file as a string. A typical session looks like this:

```
--> (compile-file "myprog.l" "myprog.bin")
"myprog.bin"

--> (load "myprog.bin")
--- [myprog.bin] loaded ---
t

--> (myfunc 42)                    ; runs through the byte-code VM
```

Once a binary file has been built, it can be loaded any number of times exactly as if it were source. There is no separate fasl-load, no version stamp, no special library directory; if the source file would have been found and loaded by **load**, its compiled form will be too.

39.6. Speed Improvement

Compiled code is roughly two times faster than interpreted code on arithmetic-intensive workloads. The exact factor depends on what the function does: code that is dominated by deep recursion and small arithmetic operations sees the largest gain, code dominated by atom look-up or large-list traversal sees less. The following measurements were taken with the V6.00 interpreter (GCC 15, x86_64) on two canonical micro-benchmarks: the doubly-recursive Fibonacci function and the triply-recursive Takeuchi function.

benchmark	interpreted	compiled	speedup
fib(28)	0.078 s	0.041 s	1.9x
fib(32)	0.532 s	0.283 s	1.88x
fib(34)	1.515 s	0.773 s	1.96x
tak(18,12,6)	0.135 s	0.099 s	1.36x

The speedup comes primarily from eliminating the per-form work done by **eval**: each call no longer walks the source list, dispatches on **celltype**, or allocates an evlis'd argument list; instead it executes a small sequence of byte codes against a dedicated stack. Operations that are already expensive per call (string interning, hash lookup, hunk allocation, large list copying) cost the same in both modes, so the proportional benefit shrinks as the per-call work grows.

A useful rule of thumb: compile any function you intend to call in a tight loop, recursively to non-trivial depth, or repeatedly from within other compiled code. One-shot top-level forms, simple configuration code, and macros that are expanded only at edit time can remain interpreted.

39.7. When To Compile

You should also be aware of two interactions with the rest of the language.

Functions defined with **defmacro** should be compiled with care. Macros run at compile time, so a macro that is itself compiled will execute through the byte-code interpreter during the compilation of any caller. This is correct but slower than leaving the macro interpreted, particularly for short macros.

39.8. Limitations

The compiler does not perform any kind of type analysis, register allocation, or function inlining beyond the constant folding mentioned above. It is also not a native-code compiler: the output is byte code that runs on a software interpreter. Closures, variadic **lexpr** bodies, and full throw**catch/** control flow are all supported, as are all of the special forms recognised by **eval**. There is no provision for saving and restoring a complete LISP image; to recreate a configured environment at startup, place the necessary forms in a file, compile it with **compile-file**, and have your **pclisprc.l** load the result.